1) Evaluation Criteria

Error image:

$$\mathcal{E}(f, \tilde{f}) = f - \tilde{f}$$

Mean Square Error (MSE):

$$\mathcal{D}(f, \tilde{f}) = \frac{1}{NM} \|\mathcal{E}\|^2$$

Peaks Signal to Noise Ratio (PSNR)

$$PSNR(f, \tilde{f}) = 10 \log_{10} \frac{255^2}{\mathcal{D}(f, \tilde{f})}$$

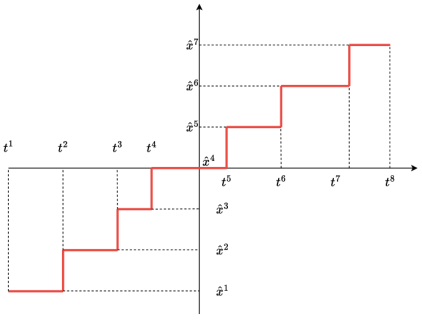
2) Quantization

The quantization is the process of mapping a function Q from $\mathbb{R}$ to a discrete set called *Dictionary*.

$$Q : x \in \mathbb{R} \rightarrow y \in C = \{\hat{x}_1, \hat{x}_2, \dots\} \subset \mathbb{R}$$

with:

- C: dictionary (subset of R)
- $\hat{x}_i$: quantization level, codeword

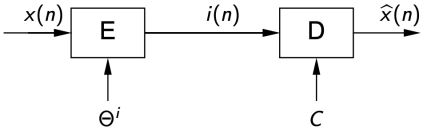


Quantizer

Moreover we define:

- Thresholds (t_1, t_n)
- Levels: $L = n - 1$
- Region Θ_i: Two subsequent thresholds define a region $\Theta_i = (t_i, t_{i+1}) = \{x : Q(x) = \hat{x}_i\}$ these are a partition (non intersecting intervals)
- Quantization error: $e = x - Q(x)$. The smaller the regions, the better the error.

Quantization can be seen as an encoding/decoding process:
take signal $x(n)$: the encoding step assigns each sample of $x(n)$ to a quantization level $i(n)$. The decoder associates to each quantization level $i(n)$ the corresponding code word (value).



Encode/Decode

We will refer to these steps also as quantization and inverse quantization.

Rate R:
The rate is the avg amount of bits used to store the quantization indexes $i(n)$. This is saved via lossless coding:

$$R = \log_2 L$$

- quantized data has uniform (pessimistic) distribution (since $H \leq R \iff$ signal uniform)
- binary data of $i(n)$ is the best (entropy coded)

Distortion D:
On a single sample we use

$$d[x, Q(x)] = \| e \|^2 = \|x - Q(x)\|^2$$

For a signal of duration N we use the MSE:

$$D = \frac{1}{N}\sum_{n=0}^{N-1} d[x(n), Q\big(x(n)\big)]$$

for a random signal this can be done via an expected value:

$$D = \mathbb{E}\left[\|X(n) - Q(X(n))\|^2\right] = \mathbb{E}\left[\|E(n)\|^2\right] = \sigma_E^2$$

Mid treat quantizers are quantizers where values near 0 are have quantization level $= 0$. This reduces noise. **Mid rise** quantizers amplify noise!

2.1) Uniform Quantizer (UQ)

Input signal $\in (0, A)$ (unsigned) or $\in (-\frac{A}{2}, \frac{A}{2})$ (signed) is divided into L equal sized cells of size $\Delta = A/L$. Where the quantization levels $\hat{x}_i$ is the midpoint. clearly this is now **mid rise**

$$\begin{gathered}
\forall i,\ \Delta^i = \Delta = A/L \\
t^i = t^{i-1} + \Delta \\
\hat{x}^i = \frac{t^i + t^{i-1}}{2} \\
\Theta^i = \left(\hat{x}^i - \frac{\Delta}{2}, \hat{x}^i + \frac{\Delta}{2}\right)
\end{gathered}$$

2.1.1) Types of UQ

Unsigned Data

For unsigned data $\in (0, A)$ we have

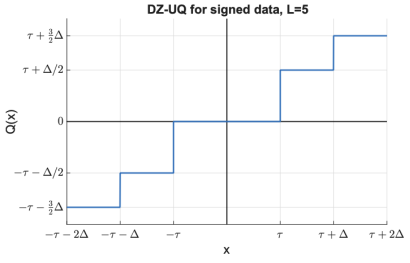
$$\begin{gathered}
\text{Thresholds: } t_i = (i-1)\Delta \\
\text{Encoder: } i = \left\lceil \frac{x}{2} \right\rceil \\
\text{Decoder: } \hat{x}^i = i\Delta - \frac{\Delta}{2} \\
\implies Q(x) = \Delta\left\lceil \frac{x}{2} \right\rceil - \frac{\Delta}{2}
\end{gathered}$$

Signed Data

Since values are unsigned a mid thread quantizer is necessary, therefore we need an **odd number of levels**. The threshold at $t^{N/2}$ will have $\hat{x}^{N/2} = 0$.

$$\begin{gathered}
\text{Encoder: } i = \text{round}\left(\frac{x}{2}\right) \\
\text{Decoder: } \hat{x}^i = i\Delta \\
\implies Q(x) = \Delta\text{round}\left(\frac{x}{2}\right)
\end{gathered}$$

From here we can define **deadzone quantization**:
The central cell is larger than the others, it is zero if $\|x\| < \tau$



Deadzone UQ

$$i = \begin{cases} sign(x) \left\lfloor \frac{\| x \| + \frac{\tau\Delta}{2}}{\Delta} \right\rfloor & \| x \| \geq \tau \\ 0 & \| x \| < \tau \end{cases}$$

High Resolution (HR) UQ

For HR we mean a quantizer with $L \to \infty$. Therefore we get $\Delta = A/L \to 0$ and therefore Θ_i as $\Theta_i = (\hat{x}_i - \Delta/2, \hat{x}_i + \Delta/2) \to (\hat{x}_i, \hat{x}_i)$. Now we can approximate p_X as a constant in each Θ_i.

Therefore in a single region:

$$E|X \in \Theta_i = X - Q(X) = X - \hat{x} \sim \mathcal{U}(-\frac{\Delta}{2}, \frac{\Delta}{2})$$

Due to total probability law we have:

$$E \sim \mathcal{U}\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$$

2.1.2) RD Curve

Probability Theory Recap

First recall the definition of uniform RV:

$$X \sim \mathcal{U}(a,b) \iff f_X(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 \end{cases} \qquad \begin{aligned} \mathbb{E}[X] &= \frac{a+b}{2} \\ \text{var}(X) &= \frac{(b-a)^2}{12} \end{aligned}$$

and also the Law of the Unconscious Statistician (LOTUS):

$$E[g(x)] = \int g(x) f_X(x) dx$$

Uniform RV with UQ

Hypothesis: $X \sim \mathcal{U}(-\frac{A}{2}, \frac{A}{2})$ quantized with UQ of L levels.
Goal: Since the rate is known ($R = \log_2 L$) we must only find the distortion: $D = \sigma_Q^2 = \mathbb{E}[\|E\|^2] = \mathbb{E}\left[\left\|X - \hat{X}\right\|^2\right]$.
Result:

$$\boxed{D(R) = \sigma_Q^2 = \sigma_X^2 2^{-2R}}$$

Proof:
Notice that since $\mathbb{E}\left[\left\|X - \hat{X}\right\|^2\right] = \mathbb{E}[g(X)]$ we can use LOTUS:

$$\begin{aligned} \sigma_Q^2 = \mathbb{E}[g(X)] &= \int_{-\frac{A}{2}}^{\frac{A}{2}} g(u) \frac{1}{A} du = \sum_{i=1}^{L} \int_{\Theta_i} \frac{1}{A} [u - Q(u)]^2 du \\ &= \frac{1}{A} \sum_{i=1}^{L} \int_{\hat{x}_i - \Delta/2}^{\hat{x}_i + \Delta/2} [u - \hat{x}_i]^2 du \\ &= \frac{1}{A} \sum_{i=1}^{L} \int_{-\Delta/2}^{\Delta/2} t^2 dt \\ &= \frac{A}{L} \frac{\Delta^3}{12} \end{aligned}$$

finally recall that in UQ we have $\Delta = A/L$, that $L = 2^R$ and that $\text{var}(X) = A^2/12$ then:

$$\sigma_Q^2 = \frac{A}{L} \frac{\Delta^3}{12} = \frac{\Delta^2}{12} = \frac{1}{12} \left(\frac{A}{L}\right)^2 = \frac{A^2}{12} \frac{1}{L^2} = \sigma_X^2 2^{-2R}$$

and the SNR becomes:

$$SNR = 10\log_{10}\frac{\mathbb{E}[X^2]}{D} = 10\log_{10}\frac{\sigma_X^2}{\sigma_X^2 2^{-2R}} = 10\log_{10}2^{2R} \approx 6R$$

□

HRUQ RD Curve

In this case we know $E \sim \mathcal{U}\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$ and thus

$$D = \mathbb{E}[\|E\|^2] = \frac{\Delta^2}{12} = \frac{1}{12}\left(\frac{A}{L}\right)^2 = \frac{A^2}{12}2^{-2R}$$

The SNR becomes:

$$SNR = 10\log_{10}\frac{\mathbb{E}(X^2)}{D} = 10\log_{10}\frac{\sigma_X^2 2^{-2R}}{A^2/12} \approx 6R - 10\log_{10}\frac{\gamma^2}{2}$$

where γ^2 is the **load factor**, that is, the ratio between peak power and avg power:

$$\gamma^2 = \frac{X_{\max}^2}{\sigma_X^2} = \frac{A^2/4}{\sigma_X^2}$$

2.1.3) Recap:

Here is a tabe:

	Unsigned	**Signed**	**Deadzone (Signed)**	**High Resolution**
Type	Mid-rise	Mid-tread	Mid-tread (large center)	Any ($L \to \infty$)
Implementation: $Q(x)$	$\Delta \cdot \left\lfloor \frac{x}{\Delta} \right\rfloor + \frac{\Delta}{2}$	$\Delta \cdot \text{round}\left(\frac{x}{\Delta}\right)$		$\Delta \cdot \text{round}\left(\frac{x}{\Delta}\right)$
L	Even	Odd	Odd	Any
RD Curve	-	if uniform input $\sigma_X^2 2^{-2R}$		$\frac{\gamma^2}{3}\sigma_X^2 2^{-2R}$

Given range and levels, UQ has the **smalles maximum error** (optimal minimax quantizer) of $e_{\max} = \frac{\Delta_i}{2} = \frac{A}{2L}$

Only quantizer with analytical solution for RD curve with uniform input distrbution (see Uniform RV with UQ):

$$D(R) = \sigma_X^2 2^{-2R}$$

in the general case (no knowledge on input distribution):

$$D(R) = \frac{A^2}{12}2^{-2R}$$

this is used for HRUQ also. Recall γ^2 is max power over avg power and $c_X = \frac{\gamma^2}{3}$.

2.2) Optimal Scalar Quantization (SQ)

Let the input density p_X be known. We want to find the quantizer minimizing the distortion (for a given rate).

Optimal HR Quantizer

Recall the HR hypothesis:

$$L \to \infty \quad \max_i \Delta_i \to 0 \quad \forall i, u \in \Theta^i,\ p_X(x) \approx P_i$$

It can be shown that the optimal quantizer is:

$$\sigma_Q^2 = c_X \sigma_X^2 2^{-2R} \qquad \text{with } c_X = \frac{1}{12}\left[\int p_U^{1/3}(t)dt\right]^3 \text{ and } U = \frac{X}{\sigma_X}$$

The term c_X is called **shape factor** since it only depends on the PDF shape and not variance (U and X have same shape)
some common shape factors are:

- Uniform: $c_X = 1$
- Gaussian $c_X = \frac{\sqrt{3}}{2}\pi \approx 2.72$

Non-HR Optimal Quantization (TODO)

There is **no analytical formula** for low rate optimal quantizers, but it is possible to find **necessary conditions** that allow to defining the **Lloyd-Max algorithm** to find local optimum points

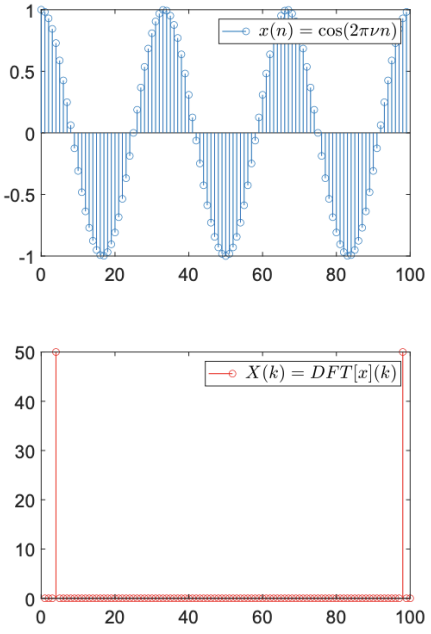
2.3) Predictive SQ

Quantization is **not effective for non sparse data**. Predictive SQ **exploits correlation among samples**.

Definition 1 (Sparse Signal)**.**

A signal is sparse if most of its components are zero or close to zero

- The variance of a sparse signal is low
- Zero (or close to zero) samples can be neglected (quantized to 0) without introducing distortion
- Natural signal are not sparse but can easily be transformed into one



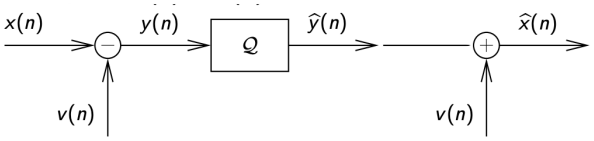
Example non sparse vs sparse representation of sinusoidal

Optimization Paradigm

Since prediction error = signal error, then performance increase depends only by predictor variance, that is **if and only if the pred error has smaller variance than og signal**

Proof prediction error=signal error:

$$q(n) = y(n) - \hat{y}(n) = x(n) - v(n) - (\hat{x}(n) - v(n)) = x(n) - \hat{x}(n) = \overline{q}(n)$$



Quantizer process

Proof SNR quality depends on σ_Y^2

$$SNR = 10\log_{10}\frac{\sigma_X^2}{D} = 10\log_{10}\frac{\sigma_X^2}{\sigma_Y^2} + 10\log_{10}\frac{\sigma_Y^2}{D} = G_P + G_Q$$

Predictors

Linear predictors are used: simple and optimal for gaussian rvs.

A linear predictor is a linear combination of the P previous values

$$v(n) = -\sum_{i=1}^{P} a_i x_{n-i} \rightarrow y(n) = x(n) - v(n) = \sum_{\mathbf{i=0}}^{P} a_i x_{n-i}, \quad a_0 = 1$$

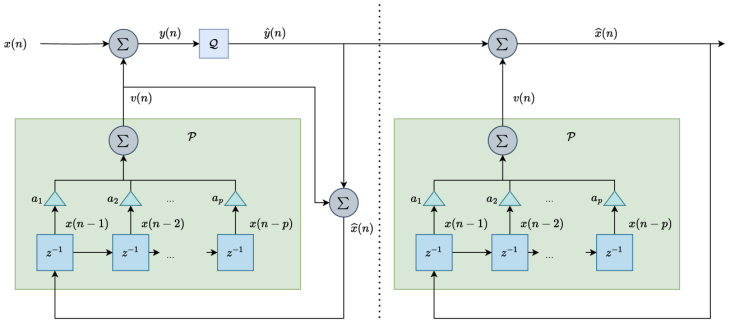
It turns out that the optimal variance is:

$$\sigma_Y^2 = \sigma_X^2 + r^T a^* = \sigma_X^2 - r^T R_X^{-1} r$$

where $a^* = -R_X^{-1} r$

The order of the filter yields good result for small values (up to order 4), but including distant pixels (less correlation) leads to more white noise inclusion and has higher complexity for negligible PSNR increases.

Clearly now v_n must use $\hat{x}$ since otherwise the decoder wouldn't be able to use know v.



Block Sceme

The use of $\hat{x}$ instead of x comes with some heavy performance penalty, it performs worse than direct quantization. This can be fixed via entropy coding.

Finally, a **local adaptation with block wise approach** can be used. Divide image into $M \times M$ blocks and find the optimal filter for each block. The rate augments by $\frac{NB}{M^2}$:

- Small block: good local statistichs, bad quantizer info overhead (high rate)
- Large block: bad local statistics (bad PSNR), good quantizer overhead

Example.

Let $X(n) \sim \mathcal{N}(0, \sigma^2), \mathbb{E}[X(n)X(m)] = \sigma^2 \rho^{|n-m|}$ *and* $V(n) = X(n-1)$. *Find* ρ *such that* $G_p > 0$.

Since we need $G_p > 0$ *we actually need* $\sigma_Y^2 > \sigma^2$.
Find Y

$$Y(n) = X(n) - V(n) = X(n) - X(n-1)$$

since this is a sum of gaussians this remains a zero mean gaussian rv, find variance:

$$\begin{aligned}\sigma_Y^2 &= \mathbb{E}[(X(n) - X(n-1))^2] = \mathbb{E}[X(n)^2] + E[X(n-1)^2] - 2\mathbb{E}[X(n)X(n-1)] \\ &= 2\sigma^2 - 2\sigma^2\rho \\ &= 2\sigma^2(1-\rho)\end{aligned}$$

$$\sigma_Y^2 = 2\sigma^2(1-\rho) > \sigma^2 \iff \rho > \frac{1}{2} \iff G_p > 0$$

Proof of optimal filter:
Notice that the predicted output is

$$y(n) = (a * x)(n) \xrightarrow{z} Y(z) = A(z)X(z)$$

where the transfer function is

$$A(z) = \sum_{i=0}^{P} a_i z^{-i} = 1 + a_1 z^{-1} + \ldots + a_P z^{-P}$$

It is clear that the minimization problem only acts on finding a that minimizes the variance.

In general the variance can be written as:

$$\sigma_Y^2 = \sigma_X^2 + 2r^T a + a^T R_X a$$

where:

- r is the autocorrelation vector
- R_X the autocorrelation matrix

The minimization can be computed:

$$\frac{\partial \sigma_Y^2}{\partial a} = 2r + 2R_X a = 0 \rightarrow a^* = -R_X^{-1} r$$

Then the optimal variance becomes:

$$\sigma_Y^2 = \sigma_X^2 + r^T a^* = \sigma_X^2 - r^T R_X^{-1} r$$

2.4) Chapter Recap

Fundamentals:
Quantizer: map $Q : x \in \mathbb{R} \rightarrow \hat{x}_i \in C = \{\hat{x}_1, \hat{x}_2, \ldots\}$. Defined by thresholds t_i, regions $\Theta_i = (t_i, t_{i+1}) = \{x : Q(x) = \hat{x}_i\}$, levels $\hat{x}_i$. $L =$ #levels. Error $e = x - Q(x)$.

Seen as encode ($x \rightarrow i$) / decode ($i \rightarrow \hat{x}_i$, = inverse quantization).

- **Rate:** $R = \log_2 L$. Assumes index distribution uniform (pessimistic, since $H \leq R$ with equality iff uniform) and binarization is entropy-coded.
- **Distortion (MSE):** $D = \mathbb{E}\left[\|X - Q(X)\|^2\right] = \sigma_E^2$.
- **Mid-tread** has a 0 level $\Rightarrow$ suppresses near-zero noise; needs **odd** L. **Mid-rise** has no 0 level $\Rightarrow$ amplifies noise; **even** L.

Uniform Quantizer (UQ)
$\Delta = A/L$ constant $\forall i$. Levels = cell midpoints, $\Theta_i = \left(\hat{x}_i - \frac{\Delta}{2}, \hat{x}_i + \frac{\Delta}{2}\right)$.

Unsigned	Signed	Deadzone (signed)	High Resolution	
Range	$(0, A)$	$(-\frac{A}{2}, \frac{A}{2})$	$(-\frac{A}{2}, \frac{A}{2})$	any
Type	Mid-rise	Mid-tread	Mid-tread (large center)	any, $L \to \infty$
L	even	odd	odd	$\to \infty$
$Q(x)$	$\Delta \left\lfloor \frac{x}{\Delta} \right\rfloor + \frac{\Delta}{2}$	$\Delta, \text{round}\left(\frac{x}{\Delta}\right)$	enc/dec with threshold τ (below)	$\Delta, \text{round}\left(\frac{x}{\Delta}\right)$
$D(R)$	—	$\sigma_X^2 2^{-2R}$ (uniform input)	—	$\frac{\gamma^2}{3}\sigma_X^2 2^{-2R}$

Deadzone (DZQ): central cell wider, $\hat{x} = 0$ for $\|x\| < \tau$. Common in compression.

$$i = \begin{cases} \text{sign}(x) \left\lfloor \frac{\|x\| + \frac{\tau\Delta}{2}}{\Delta} \right\rfloor & \|x\| \geq \tau \backslash [2pt] 0 \quad \|x\| < \tau \end{cases} \qquad \hat{x} = \begin{cases} \text{sign}(i), \Delta\left(\|i\| + \frac{1-\tau}{2}\right) & i \neq 0 \backslash 0 \quad i = 0 \end{cases}$$

Key results

- **Minimax optimal:** given range and L, UQ has the smallest max error $e_{\max} = \frac{\Delta}{2} = \frac{A}{2L}$.
- Only quantizer with **analytical RD** under uniform input: $\boxed{D(R) = \sigma_X^2 2^{-2R}}$, with SNR $\approx 6R$.
- General case (HR, no input knowledge): $\boxed{D(R) = \frac{A^2}{12}2^{-2R} = \frac{\gamma^2}{3}\sigma_X^2 2^{-2R} = K_X \sigma_X^2 2^{-2R}}$

$$\text{SNR} = 10\log_{10}\frac{\sigma_X^2}{A^2/12}2^{2R} \approx 6R - 10\log_{10}\frac{\gamma^2}{3}, \qquad \gamma^2 \overset{\triangle}{=} \frac{X_{\max}^2}{\sigma_X^2} = \frac{A^2/4}{\sigma_X^2}$$

where γ^2 = **load factor** (peak power / avg power) and $K_X = \gamma^2/3$.

> **Definition 2** (HR hypothesis $L \to \infty \Rightarrow \Delta \to 0 \Rightarrow p_X(x) \approx P_i$ const on each Θ_i. Then $E \mid X \in \Theta_i \sim \mathcal{U}(-\frac{\Delta}{2}, \frac{\Delta}{2})$, and by total probability $E \sim \mathcal{U}(-\frac{\Delta}{2}, \frac{\Delta}{2}) \Rightarrow D = \frac{\Delta^2}{12}$.)**.

Optimal Scalar Quantization

Input density p_X known $\Rightarrow$ minimize D at fixed R.

Optimal HR quantizer

$$\sigma_Q^2 = c_X, \sigma_X^2, 2^{-2R}, \qquad c_X = \frac{1}{12}\left[\int p_U^{1/3}(t), dt\right]^3, \quad U = \frac{X}{\sigma_X}$$

c_X = **shape factor**: depends only on the PDF *shape*, not variance (U, X same shape).

- Uniform: $c_X = 1$.
- Gaussian: $c_X = \frac{\sqrt{3}}{2}\pi \approx 2.72$.

Non-HR (low rate)
No analytical formula. Necessary optimality conditions ⇒ **Lloyd–Max** algorithm → local optimum.

Predictive SQ

Quantization is ineffective for non-sparse data. Predictive SQ exploits **inter-sample correlation**.

Definition 3 (Sparse Signal Most components zero or near zero.)**.**

- Low variance.
- Near-zero samples quantized to 0 with negligible distortion.
- Natural signals aren't sparse, but can be sparsified (prediction, linear transform).

Optimization paradigm
Prediction error and signal error coincide ⇒ performance gain depends only on predictor variance: prediction helps **iff** $\sigma_Y^2 < \sigma_X^2$.

Proof (prediction error = signal error), valid in **closed loop** (v built from $\hat{x}$ at both ends):

$$q(n) = y(n) - \hat{y}(n) = \big(x(n) - v(n)\big) - \big(\hat{x}(n) - v(n)\big) = x(n) - \hat{x}(n) = \overline{q}(n)$$

SNR splits into prediction gain + quantizer gain:

$$\text{SNR} = 10\log_{10}\frac{\sigma_X^2}{D} = \underbrace{10\log_{10}\frac{\sigma_X^2}{\sigma_Y^2}}_{G_P} + \underbrace{10\log_{10}\frac{\sigma_Y^2}{D}}_{G_Q}$$

Linear predictor
Simple, optimal for Gaussian rv. Combination of P past values:

$$v(n) = -\sum_{i=1}^{P} a_i x_{n-i} \;\Rightarrow\; y(n) = x(n) - v(n) = \sum_{i=0}^{P} a_i x_{n-i}, \quad a_0 = 1$$

As a filter: $y(n) = (a * x)(n) \xrightarrow{\mathcal{Z}} Y(z) = A(z)X(z)$, with $A(z) = \sum_{i=0}^{P} a_i z^{-i} = 1 + a_1 z^{-1} + \cdots + a_P z^{-P}$.
Minimization acts only on a.

Optimal filter (Yule–Walker): the variance is a quadratic form

$$\sigma_Y^2 = \sigma_X^2 + 2r^T a + a^T R_X a, \qquad r = \text{autocorr. vector},\ R_X = \text{autocorr. matrix}$$

$$\frac{\partial \sigma_Y^2}{\partial a} = 2r + 2R_X a = 0 \;\Rightarrow\; \boxed{a^* = -R_X^{-1} r} \;\Rightarrow\; \boxed{\sigma_{Y,\min}^2 = \sigma_X^2 + r^T a^* = \sigma_X^2 - r^T R_X^{-1} r}$$

Practical points

- **Order:** good up to ≈ 4. Distant pixels add little (screening effect: their correlation is already captured by closer neighbors) at higher complexity, negligible PSNR gain.
- **Closed loop mandatory:** predictor must use $\hat{x}$, else $v_{enc} \neq v_{dec}$, the cancellation $q = \overline{q}$ breaks, and error accumulates (**drift**).

- **Penalty:** predicting from $\hat{x}$ (not x) performs worse than direct quantization; recovered via **entropy coding**.
- **Local (block-wise) adaptation:** split image into $M \times M$ blocks, optimal filter per block. Rate overhead $+\frac{NB}{M^2}$ (N=order, B=bits/coeff).
  - Small block: good local stats, high overhead.
  - Large block: low overhead, poor local adaptation.

Example (Prediction gain for AR(1) $X(n) \sim \mathcal{N}(0, \sigma^2), \mathbb{E}[X(n)X(m)] = \sigma^2 \rho^{\|n-m\|}$, fixed predictor $V(n) = X(n-1)$. Find ρ s.t. $G_P > 0$.)**.

Need $G_P > 0[\text{dB}] \iff \sigma_Y^2 < \sigma_X^2 = \sigma^2$. *With* $Y(n) = X(n) - X(n-1)$ *(zero-mean Gaussian):*

$$\sigma_Y^2 = \mathbb{E}[(X(n) - X(n-1))^2] = 2\sigma^2 - 2\sigma^2\rho = 2\sigma^2(1-\rho)$$

$$\sigma_Y^2 < \sigma^2 \iff 2(1-\rho) < 1 \iff \boxed{\rho > \tfrac{1}{2}} \iff G_P > 0$$

3) Lossless Coding

Lossless coding means to decrease the number of bits needed to encode the data without losing any information: the process is reversible.

This is used in the quantization part: remap the indices $i(n)$ in order to optimize bitstream.

We must understand:

- **Theoretical Bound:** Source entropy
- **Practical Implementation:** algorithm efficiency and scalability
- **Possible without known statistics?**

Lets start with some notation:

- **Alphabet:** $\mathcal{X} = \{x_1, \dots, x_M\}$ set of symbols to encode
- **Code:** application between $\mathcal{X}$ and $\{0, 1\}$ (set of finite length bit strings)

Fixed Length Coding (FLC)

FLC assumes equiprobable alphabet: all codewords ave the same length, that is M symbols $\rightarrow L = \lceil \log M \rceil$ bits to encode each symbol (bpS) and a rate of $R = \log_2 L$

Example (FLC Text Compression)**.**

An alphabet with 26 symbols is encoded with $\lceil \log 26 \rceil = \lceil 4.7 \rceil = 5$ *bits per codeword. This has a compression ratio of 1.*

Variable Length Code (VLC)

Exploit input distribution to assign less bits to more probable symbols.
Parsing problem: create code where symbol is perfectly distinguishable in the bitstream

3.2) Principles of Information Theory

It is clear that there is a preferred code. VLC has a **prefix condition** where a instantaneous (prefix) code has no codeword is a prefix of another codeword.

Theorem 4 (McMillan's Theorem)**.**

Decodable codes do not improve performance with respect to instantaneous codes

Therefore, **we can focus only on instantaneous codes**

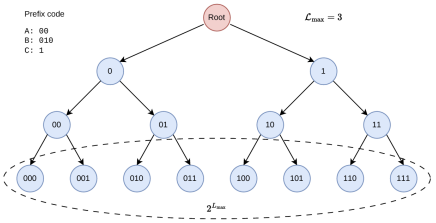
Theorem 5 (Kraft's Inequality)**.**

There exists a instantaneous code with lengths $\{l_1, \dots, l_M\}$ *iff*

$$\sum_i 2^{-l_i} \leq 1$$

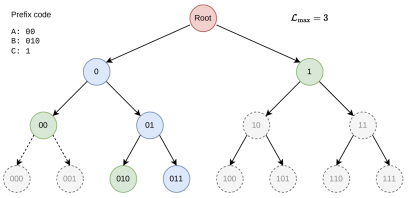
Kraft Proof:

Proof of Necessity ($\implies$): Codewords are already given, we must show the formula
Set $L_{\max} = \max_i \{l_i\}$.
Then, by building a binary tree of the possible codes, the number of leaves (at level $L_{\max}$) is $2^{L_{\max}}$.



Binary Tree Example

Each code c_i has $2^{L_{\max}-l_i}$ leaves and due to **prefix property** no codeword is a descendant of another: leaves are disjoint. The tree can be truncated at the position of each codeword



Example Tree Truncated at Codewords

Therefore we have that the number of leaves (codewords) are necessarily less than the maximum leaves:

$$\sum_{i=1}^{N} 2^{L_{\max}-l_i} \leq 2^{L_{\max}}$$

$$\sum_{i=1}^{N} 2^{-l_i} \leq 1$$

Proof of Sufficiency ($\impliedby$): We have the formula, we must build the code
The codes can be built as follows:
0. Create a complete binary tree of depth $L_{\max}$

1. Sort all lengths $l_1 \leq \ldots \leq l_N$
2. At each step $k = \{1, \ldots, N\}$ pick an available position at depth l_k (increasing order)
3. Mark descendants of l_k as forbidden. $2^{L_{\max}-l_i}$ leaves are blocked.

The code can be built if at step k there is an available codeword of length l_k.
At each step the unavailable leaves (before adding c_k) are $\sum_{i=1}^{k-1} 2^{L_{\max}-l_i}$. Therefore we have $2^{L_{\max}} - \sum_{i=1}^{k-1} 2^{L_{\max}-l_i}$ available leaves.
This can be normalized wrt the number of leaves $1 - \sum_{i=1}^{k-1} 2^{-l_i}$ available leaves.

By Kraft we have

$$\sum_{i=1}^{N} 2^{-l_i} = \sum_{i=1}^{k-1} 2^{-l_i} + \sum_{i=k}^{N} 2^{-l_i} \leq 1$$
$$\sum_{i=1}^{k-1} 2^{-l_i} \leq 1 - \sum_{i=k}^{N} 2^{-l_i}$$
$$1 - \sum_{i=1}^{k-1} 2^{-l_i} \geq \sum_{i=k}^{N} 2^{-l_i} \geq 2^{-l_k} > 0$$

Since the remaining free capacity is at least 2^{-l_k}, there must exist at least one free node at depth l_k where the k-th codeword can be placed.

□

Information and Entropy Recap

Suppose symbols $\{x_i\}$ with probabilities p_i.

- The information of a symbol is
$$I(x_i) = -\log_2 p_i = \log_2 \frac{1}{p_i}$$
- Properties:
$$I(x_i) > 0$$
$$\text{if } p_i = 1 \to I = 0$$
$$\text{if } x_i \text{ indep } x_j \to I(x_i, x_j) = I(x_i) + I(x_j)$$
- The source entropy of $X = \{x_i\}$ is
$$H(X) = -\sum_i p_i \log_2 p_i = -\mathbb{E}[\log_2(p_X(x))]$$
this is the avg uncertainty of X.
- Joint Entropy:
$$H(X,Y) = -\sum_{i,j} p_{i,j} \log p_{i,j}$$
- Conditional Entropy
$$H(X|Y) = \sum_j p_j H(X|Y=y_j) \to H(X,Y) = H(Y) + H(X|Y)$$
$$= H(Y) + H(Y|X)$$
- Properties:
$$H(X) > 0$$
$$H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$$
$$H(X,Y) \leq H(X) + H(Y) \ (= \text{if } \perp)$$
$$H(X|Y) \leq H(X) \ (= \text{if } \perp)$$
$$H(X) \leq \log_2 M \ (= \text{if } X \sim u)$$

> **Theorem 6** (Fundamental: Conditioning Reduces Entropy)**.**
>
> $$H(X|Y) \leq H(X)$$
>
> *That is: information never hurts, knowing Y we can better infer X*

Lagrange's Method

Lagrange's method is a solution for minimax problems under a constraint $\phi(x) = 0$

Consider a function $f : x \in \mathbb{R}^n \to \mathbb{R}$
In order to find the maximum or minimum of f subject to the constraint $\phi(x) = 0$, we look for the stationary points of

$$J(x, \lambda) = f(x) + \lambda\phi(x), \quad \lambda \in \mathbb{R}$$

The stationary points are computed by setting to zero all the derivatives of J:

$$\frac{\partial J}{\partial x_i} = 0, \quad \frac{\partial J}{\partial \lambda} = 0$$

Example (Distribution with Maximum Entropy)**.**

The distribution maximizing the entropy of a M-ary discrete r.v. is found applying the Lagrange's method:

$$p^* = \arg\max_p \sum_{i=1}^{M} p_i \log\frac{1}{p_1}, \qquad \sum_i p_i = 1 \to \phi(x) = \sum_i p_i - 1 = 0$$

Write J:

$$J(p.\lambda) = -\sum_i p_i \log p_i + \lambda\left(\sum_i p_i - 1\right)$$

Calculate the derivative (specific p_i)

$$\frac{\partial J}{\partial p_i} = -\left(\frac{\log e}{p_i} p_i + \log p_i\right) + \lambda = 0 \to p_i = \lambda - \log e$$

$$\frac{\partial J}{\partial \lambda} = \sum_{i=1}^{M} p_i - 1 = 0 \to p_i = 1/M$$

The max uncertainty is obtained by setting all probabilities equal, that is $p_i = 1/M$.

3.3) Optimal Code

The optimal code is the solution ot the following problem:

Given a set of M symbols with probabilities p_i, the optimal code is the set of lengths such that:

- $\forall i = 1, \dots, M, l_i \in \mathbb{N}$
- Kraft inequality is satisfied (prefix code)
- The averge length $\mathcal{L} = \sum_{i=1}^{M} p_i l_i$ is minimized among all possible sets of l_i's

The optimal solution drops the first assumption ($l_i \in \mathbb{R}$) and obtains:

$$l_i^* = -\log_2 p_i = I(x_i) \to \mathcal{L}^* = \sum_i -p_i \log_2 p_i = H(X)$$

Since real codes cannot have fractional length, thsi bound is achieved only if the distribution is dyadic, that is every symbol's probability is a negative power of 2:

$$\forall i \in \{1, \dots, M\},\ \exists k \in \mathbb{N} \ \mid \ p_i = 2^{-k}$$

therefore in the general case $\mathcal{L}^* \geq H(X)$ but by how much?

Theorem 7 (Shannon Source Coding Theorem)**.**

For any random source X, the minimum average length $\mathcal{L}^*$ achievable by an instantaneous (prefix) code satisfies:

$$\mathcal{L}^* \geq H(X)$$

*with equality if and only if the distribution of X is **dyadic**.*

Proof:
This is a constrained optimization problem:

$$l^* = \arg\min_l \sum p_i l_i \qquad \sum_i 2^{-l_i} = 1$$

$$J = \sum p_i l_i + \lambda(\sum 2^{-l_i} - 1) \rightarrow \frac{\partial J}{\partial l_i} = p_i - \lambda \cdot 2^{-l_i} \ln 2 = 0 \rightarrow p_i = \lambda \ln 2 \cdot 2^{-l_i}$$

Sum all p_i:

$$\sum_i p_i = \lambda \ln 2 \sum 2^{-l_i}$$
$$1 = \lambda \ln 2$$

by placing this result in the initial derivative

$$\frac{\partial J}{\partial l_i} = p_i - \lambda \cdot 2^{-l_i} \ln 2 = p_i - 2^{-l_i} = 0 \rightarrow p_i = 2^{-l_i} \rightarrow l_i^* = -\log_2 p_i$$

And the avg length is then just the definition of entropy

□

The equality for Kraft was used because we represent a **complete code**. One where no possible empty branch is left

Entropy Code

To keep the first assumption we have **Entropy Coding**

$$l_i = \lceil -\log_2 p_i \rceil \rightarrow H(X) \leq \mathcal{L}^* < H(X) + 1$$

Proof:
The choice of the ceil is set a priori, therefore we must first check if Kraft holds:

$$l_i = -\log_2 p_i + \delta_i \qquad 0 \leq \delta < 1$$
$$2^{-l_i} = p_i \cdot 2^{-\delta_i} = \epsilon_i p_i \qquad \frac{1}{2} < \epsilon_i \leq 1$$

$$\sum_i 2^{-l_i} = \sum_i \epsilon_i p_i \leq \sum_i p_i = 1$$

Now the lower bound on the avg length comes directly from Shannon, while the upper bound is easily verifiable:

$$l_i = \lceil -\log_2 p_i \rceil < -\log_2 p_i + 1$$
$$p_i l_i < p_i - p_i \log_2 pi$$
$$\mathcal{L} = \sum p_i l_i < \sum p_i - p_i \log_2 p_i = 1 + H(X)$$

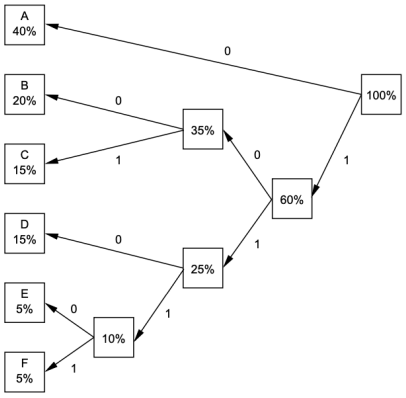
Huffman Code

Huffman discovered how to build the **optimal lossless coder for any source with known probabilities.**

Algorithm:

1. Create leaf nodes for each symbol, weighted by p_i

2. While there is more than one node:
   - Select two nodes with lowest weights
   - Create a new internal node as their parent
   - Set new node's weight as sum of children's weights
3. Assign '0' to left (upper) edges, '1' to right (lower) edges
4. Read code for each symbol from root to leaf



Example

This code has avg length $L^* = 2.3$ and entropy $H(X) = 2.246$

However now consider the following example:

Example.

Let there be a BW text image. Clearly there are many more white pixels, therefore:

$$P(X = B) = p \ll 1 \quad P(X = W) = 1 - p$$

The entropy now becomes

$$H(X) \ll 1$$

However the avg length is exactly 1.

Here the entropy is very small but the coding has high rate. This is due to a too small alphabet. It is possible to expand the alphabet by considering blocks of K symbols:

Block Coding

Instead of mapping one symbol into one codeword, we can map many subsequent symbols into a single codeword.

$$X^K = X_1 X_2 \dots X_K$$

The entropy is

$$H(X^K) \leq \sum_{i=1}^{K} H(X_i)$$

Theorem 8 (Entropic Rate)**.**

We define the Entropy Rate as

$$\mathcal{H}(X) = \lim_{K\to\infty} \frac{H(X^K)}{K}$$

For stationary processes it can be shown that

$$\mathcal{H}(X) \leq H(X)$$

Then the avg length is

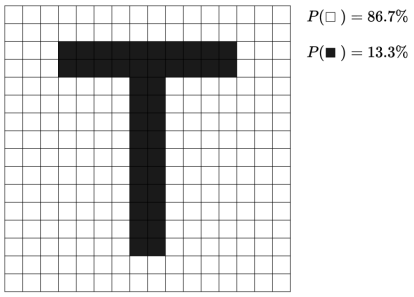
$$\begin{gathered} H(X^K) \leq \mathcal{L}^* < H(X^K) + 1 \\ \frac{H(X^K)}{K} \leq \frac{\mathcal{L}^*}{K} < \frac{H(X^K)}{K} + \frac{1}{K} \\ \mathcal{L}_S^* \to \mathcal{H}(X) \leq H(X) \end{gathered}$$

This improves the **1bit penalty for non dyadic distributions**

However the complexity is exponential with K and the joint entropy is very costly.

Example.

This example will cover the different results (calculations not shown) for the following image:



T

Block size 1: $H(X) = 0.586$, $\mathcal{L}^* = 1$
Block size 2: $H(X_1X_2) = 1.022 \to H/2 = 0.511$, $\mathcal{L}^* = 1.3 \to \mathcal{L}_S = 0.65$ *bpp*
Block size 4: $H(\prod X_i) = 1.533 \to H/4 = 0.383$, $L^* = 1.733 \to \mathcal{L}_S = 0.433$ *bpp*

Arithmetic Coding

Arithmetic coding allows to perform block coding or context-based coding with linear complexity. This coder is suboptimal but asymptotically optimal. Since it has linear complexity, it can be easier scaled.

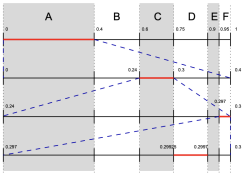
$$\mathcal{L} \leq H(X^K) + 2 \to \mathcal{L}_s = \frac{\mathcal{L}}{K} \overset{K\to\infty}{\longrightarrow} \mathcal{H}(X)$$

The code is built by determining an interval in $[0, 1]$ according to its probability. Needs only 2 sums and 2 multiplications per interval (linear complexity!).

This is done achieved by **encoding a sequence as the center interval** with arbitrary precision $q \in [0, 1]$ a fractional number. The arithmetic code can encode blocks of any size, even the entirety of the message.

Symbol	A	B	C	D	E	F
Probability	0.4	0.2	0.15	0.15	0.05	0.05

Input sequence: **ACFD**



- **Start:** $[0, 1)$
- **A:** $[0, 0.4)$
  Center: 0.2. Precision: $p_1/2 = 0.2$.
  $L(1) = -\lceil \log_2 p_1 \rceil + 1$
- **C:** $[0.24, 0.30)$
  Center: 0.275. Precision: $p_1 p_3/2$.
  $L(2) = -\lceil \log_2 p_1 + \log_2 p_3 \rceil + 1$
- **F:** $[0.297, 0.30)$
  Center: 0.2985. Precision: $p_1 p_3 p_6/2$.
  $L(3) = -\lceil \log_2 p_1 + \log_2 p_3 + \log_2 p_6 \rceil + 1$
- **D:** $[0.29925, 0.2997)$
  Center: 0.299475. Precision: $p_1 p_3 p_6 p_4/2$. $L(3) = -\lceil \log_2 p_1 + \log_2 p_3 + \log_2 p_6 + \log_2 p_4 \rceil + 1$

Example

Proof of length:
From he above image the length of a message results:

$$L(n) = 1 - \left\lceil \sum \log_2 p_i \right\rceil < 2 - \sum \log_2 p_i$$

The average message length is

$$\overline{L}(n) < \frac{2 - \sum \log_2 p_i}{n}$$

and by taking the expected value the avg length can be found:

$$\mathcal{L} = \mathbb{E}[\overline{L}(n)] < \frac{2 - \sum \mathbb{E}[\log_2 p_i]}{n} = H(X) + \frac{2}{n} \rightarrow \mathcal{L} < H(X)$$

Context Based Coding (TODO)

This approach consists in looking at N_s previous symbols to recognize the context (max $N_c = M^{N_s}$ with M the alphabet) and changes the encoder based on the context. This conditioning reduces the entropy of the source.

Engineering Challenge: with the right context a shorter number of samples manages to better reach the entropy rate.

3.3.2) Recap

Here is a brief recap

Scheme	Length rule	Average-length bound	Complexity	Key idea / limitation
Optimal (theoretical)	$l_i^* = -\log_2 p_i$ (real-valued)	$\mathcal{L}^* = H(X)$	—	Only achievable exactly if p_i is dyadic
Entropy code	$l_i^* = -\lceil \log_2 p_i \rceil$	$H(X) \leq \mathcal{L} \leq H(X) + 1$	low	Simple rounding; up to 1-bit/symbol penalty
Huffman	Greedy bottom-up tree merge	$H(X) \leq \mathcal{L} \leq H(X) + 1$	$O(M \log M)$	Provably optimal among integer-length codes; small-alphabet penalty (e.g. binary source)
Block coding	Code K symbols jointly	$L_S \rightarrow \mathcal{H}(X) \leq H(X)$ as $K \rightarrow \infty$	exponential in K (M^K symbols)	Removes 1-bit penalty asymptotically; explodes in cost
Arithmetic coding	Whole sequence → one interval/point	$\mathcal{L}_S < H(X) + \frac{2}{n} \rightarrow H(X)$	linear, $O(n)$	Sub-optimal for finite n, asymptotically optimal; scales where block coding can't
Context-based coding	Condition on N_s previous symbols	lowers effective entropy rate further	depends on $N_c = M^{N_x}$	Exploits memory/correlation; right context model is the engineering challenge

3.4) Other Techniques (TODO)

3.4.1) Exp-Golomb Coding

Universal coding for integer numbers, size (bits) proportional to magnitude

Unisgned Integer

Given int $n \in \mathbb{N}$ the representation consists in

- write $n+1$ in binary
- use min number of bits: $b = \lfloor \log_2(n+1) \rfloor + 1$
- place $b-1$ leading zeroes

Example:

$$n = 0 \rightarrow \begin{cases} n+1 = 1_{10} \rightarrow 1_2 \\ b = \lfloor \log_2 1 \rfloor + 1 = 1 \\ b - 1 = 0 \end{cases} \longrightarrow 1$$

$$n = 6 \rightarrow \begin{cases} n+1 = 7_{10} = 111_2 \rightarrow 1_2 \\ b = \lfloor \log_2 7 \rfloor + 1 = 3 \\ b - 1 = 2 \end{cases} \longrightarrow 00111$$

Signed Integer

Given int $n \in \mathbb{Z}$ the representation consists in

- Map $\mathbb{Z} \rightarrow \mathbb{N}$ using $m(n) = \begin{cases} 2n-1 & n > 0 \\ -2n & n \leq 0 \end{cases}$
- Use Exp-Golomb for unsigned integer

Example:

$$n = -3 \rightarrow m(n) = 6 \rightarrow 00111$$

$$n = -6 \rightarrow m(n) = 12 \rightarrow \begin{cases} n+1 = 13_{10} = 1101_2 \rightarrow 1_2 \\ b = \lfloor \log_2 7 \rfloor + 1 = 4 \\ b - 1 = 3 \end{cases} \longrightarrow 0001011$$

3.5) Standards (TODO)

JBIG

JPEG-LS

PNG

3.6) Neural Lossless Coding (TODO)

4) Transform Coding

Until now we have seen the following pipeline:

$$\text{Data} \overset{\text{lossy}}{\longrightarrow} \text{Quantized data at fixed rate} \overset{\text{lossless}}{\longrightarrow} \text{Entropy Coded Data}$$

The real pipeline inserts two stages before quantization:

- **Transform coding:** a reversible transform that sparsifies the block and gives each sample position a stationary meaning (e.g. DCT coefficient 0 = mean energy, the rest = increasing frequencies).
- **Block coding:** find the optimal rate per sample, that is, how sensible should be the quantizer for each sample.
- **Quantize:** apply those quantizers, this exploits unequal variances (high variance, many bits in quantizer, low variance only a few bits)
- **Entropy code:** pack the resulting bitstream exploiting the many resulting zeros to approach the entropy.

We start with **block coding**, because it defines *what we want from a transform and why*. But it must be clear from the start that block coding alone is useless on raw natural images (see Example (ID RVs (PCM))), for two distinct reasons:

- **No variable rate:** all pixels have (approximately) equal variance, so we revert to UQ
- **Too many quantizers:** even with unequal variances, one fixed quantizer can serve every block only if each position's variance is stationary across blocks (a constant per-sample meaning, as the transform provides, is one way to guarantee this).

The transform is precisely what supplies both: variance disparity (gain) and cross-block stationarity (one shared quantizer).

Digression: Arithmetic Mean (AM) vs Geometric Mean (GM)

First we must clarify the difference between AM and GM:

$$z_{AM} = \frac{1}{M}\sum_{k=1}^{M} z_k \qquad z_{GM} = \sqrt[M]{\prod_{k=1}^{M} z_k}$$

We will see that in orthonormal transforms

- AM is unchanged ($\propto \mathcal{L}^2$ norm)
- GM can be reduced (energy concentration)

By **Jensen's inequality:**

$$z_{GM} \leq z_{AM}$$

Proof
Jensens inquality states that:

$$\sum_{k=1}^{M} \frac{1}{M} f(z_k) \leq f\left(\sum_{k=1}^{M} \frac{1}{M} z_k\right) = f(z_{AM})$$

with f the log function:

$$\sum \frac{1}{M} \log z_k = \log\left(\prod z_k^{\frac{1}{M}}\right) = \log(z_{GM}) \leq \log(z_{AM})$$

□

4.2) Block Coding

Block coding aims to solve the **resource allocation problem**, that is, find the rate vector $R = [R_1, \dots, R_M]$ that minimizes global distortion on block $X = [X_1, \dots, X_M]$ under the constraint $\sum R_k = R_{tot}$. That is a different quantizer per sample

Recall that an optimal HR quantizer has a distortion for a sample of $D_k = c_k \sigma_k^2 2^{-2R_k}$. The **global distortion** is:

$$\begin{aligned}\mathcal{D} &= \frac{1}{M}\mathbb{E}[\|X_Q(X)\|^2] = \frac{1}{M}\mathbb{E}[(X - Q(X))^T(X - Q(X))] \\ &= \frac{1}{M}\mathbb{E}[\sum_k (X_k - QX_k)^2] = \frac{1}{M}\sum_k \mathbb{E}[(X_k - Q(X_k))^2] = \frac{1}{M}\sum_k D_k \\ &= \frac{1}{M}\sum_{k=1}^{M} c_k \sigma_k^2 2^{-2R_k}\end{aligned}$$

The solution is the **Huang-Schulteiss formula:**

$$R_k^* = \frac{R_{tot}}{M} + \frac{1}{2}\log_2\left[\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}\right]$$

That is, the rate is a refinement of a uniform resource allocation $\overline{R} = R_{tot}/M$.

The per sample distortion becomes:

$$D_k^* = c_k\sigma_k^2 2^{-2\overline{R} - \log_2(\cdot)} = c_k\sigma_k^2 2^{-2\overline{R}} \cdot \frac{c_{GM}\sigma_{GM}^2}{c_k\sigma_k^2} = c_{GM}\sigma_{GM}^2 2^{-2\overline{R}}$$

And the global distortion only depends on the geometric mean:

$$\mathcal{D}^* = \frac{1}{M}\sum_k D_k^* = c_{GM}\sigma_{GM}^2 2^{-2\overline{R}}$$

Here are two examples:

Example (Gaussian Optimal Rate)**.**

Recall that for a gaussian rv the shape factor is constant and has this relation: $c_{GM} = x_k = c_{\mathcal{N}}$, then clearly:

$$\mathcal{D}^* = c_{\mathcal{N}}\sigma_{GM}^2 2^{-2\overline{R}}$$

Example (ID RVs (PCM))**.**

When the k rvs are id we have that shape factor and vatiance are cnstant, therefore:

$$c_k = c_X = c_{GM} \qquad \sigma_k^2 = \sigma_X^2 = \sigma_{GM}^2$$

Then

$$R_k^* = \overline{R} + \frac{1}{2}\log\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2} = \overline{R} \quad \mathcal{D}^* = c_X\sigma_X^2 2^{-2\overline{R}}$$

the signal is not sparse at all, so block coding does not work

If the ID signals are zero mean gaussian rvs, then we call it PCM, the coding distortion becomes

$$\mathcal{D}_X = c_{\mathcal{N}}\sigma_{GM}^2 2^{-2\overline{R}}$$

Proof (TODO)

The function to minimize and the constraint are:

$$\mathcal{D} = \frac{1}{M}\sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} \qquad \sum_{k=0}^{M-1} R_k \leq R_{tot}$$

By lagrange method we must minimize:

$$J(R,\lambda) = \frac{1}{M}\sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} + \lambda\left(\sum_{k=0}^{M-1} R_k - R_{tot}\right)$$

The derivatives become:

$$\frac{\partial J}{\partial R_k} = -\frac{2\ln 2}{M} c_k \sigma_k^2 2^{-2R_k} + \lambda = 0 \qquad \frac{\partial J}{\partial \lambda} = \sum_{k=0}^{M-1} R_k - R_{tot} = 0$$

from the first we get:

$$R_k = \frac{1}{2}\log_2(c_k\sigma_k^2) + \underbrace{\frac{1}{2}\log_2\frac{2\ln 2}{M\lambda}}_{\text{constant: } \lambda'}$$

Plugging it into the second we have:

$$\begin{aligned} R_{tot} = M\lambda' + \sum \frac{1}{2}\log_2(c_k\sigma_k^2) \to \lambda' &= \frac{R_{tot}}{M} - \frac{1}{2M}\sum \log_2(c_k\sigma_k^2) \\ &= \overline{R} - \frac{1}{2}\log_2 \prod (c_k\sigma_k^2)^{1/M} \\ &= \overline{R} - \frac{1}{2}\log_2(c_{GM}\sigma_{GM}^2) \end{aligned}$$

plug it again in the initial formula:

$$R_k^* = \frac{1}{2}\log_2(c_k\sigma_k^2) + \lambda' = \overline{R} + \frac{1}{2}\log_2\frac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}$$

□

Practical Implementation of Huang-Schulteiss

HS has a series of limitations:

- Returns negative values
- Returns non-integer values

The **Modified HS algorithm** is very simple:

1. Use HS with all M components
2. If there are negative values, set $R = 0$ of the component and recalculate HS with $M - N$ components
3. When there are only positive values, floor the results
4. Calculate residual rate and allocate eventual residual bits to the results that got rounded the most

Example.

Let there be a gaussian input with 4 components and $R_{tot} = 10$:

$$\sigma_1^2 = 1000,\ \sigma_2^2 = 100,\ \sigma_3^2 = 50,\ \sigma_4^2 = 1$$

The GM is ≈ 47.29
We get:

$$R(1) \approx 4.7,\ R(2) \approx 3.04,\ R(3) \approx 2.54,\ R(4) \approx -0.28 \overset{\text{negative}}{\longrightarrow} 0$$

Recompute HS with 1,2,3:
GM ≈ 171
We get (also floor):

$$R(1) \approx 4.61 \to 4,\ R(2) \approx 2.95 \to 2,\ R(3) \approx 2.45 \to 2,\ R(4) = 0 \to 0$$

Since $4+2+2+0 \neq 10$ *we can allocate 2 residual bits to components 1 and 2*

$$R(1) = 5,\ R(2) = 3,\ R(3) = 2,\ R(4) = 0$$

The total distortion is:

$$D = \sum D_i = \sum \sigma_k^2 2^{-2R_k} = 0.98 + 1.56 + 3.13 + 1 = 6.67$$

With equal rate it was:

$$D = \sum \sigma_k^2 2^{-2 \cdot 2.5} = 35.97$$

The **greedy algorithm** returns the same allocation but is faster:

- Initialization. $R_k = 0\ \forall k \in \{0, \dots, M-1\} \qquad D_k = \sigma_k^2 \forall k \in \{0, \dots, M-1\}$
- While $\sum R_k \leq R_{tot}$
  - $l = \arg\max_k D_k$
  - $R_l \leftarrow R_l + 1$
  - $D_l \leftarrow D_l/4$ (since $(D(R+1) = \sigma_k^2 2^{-2(R+1)} = \frac{1}{4}\sigma_k^2 2^{-2R} = D(R)/4)$)
    This takes R_{tot} iterations

Example.

Consider the same data as before:

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
0	-	0	0	0	0	1000.00	100.00	50.00	1.00
1	1	1	0	0	0	250.00	100.00	50.00	1.00
2	1	2	0	0	0	62.50	100.00	50.00	1.00
3	2	2	1	0	0	62.50	25.00	50.00	1.00
4	1	3	1	0	0	15.63	25.00	50.00	1.00
5	3	3	1	1	0	15.63	25.00	12.50	1.00

Iterations 1-5

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
6	2	3	2	1	0	15.63	6.25	12.50	1.00
7	1	4	2	1	0	3.91	6.25	12.50	1.00
8	3	4	2	2	0	3.91	6.25	3.13	1.00
9	2	4	3	2	0	3.91	1.56	3.13	1.00
10	1	5	3	2	0	0.98	1.56	3.13	1.00

Iterations 6-10

4.3) Transform Coding

A non-sparse signal codes badly. This chapter looks for a **transform that sparsifies** the signal (few large coefficients, many small ones).

We want a transform with the following properties:

- **Invertible:** $Y = T(X) \iff X = T^{-1}(Y)$
- **Input in** $\mathbb{R}^M$ (e.g. an image lives in $\mathbb{R}^2$)
- **Non-sparse input → sparse output, at the same quantization error**

We restrict to a **linear** transform $Y = \mathcal{T}X$, with $\mathcal{T}$ an invertible matrix:

- inverse exists by definition;
- it acts as a **change of basis** (the columns of $\mathcal{T}^{-1}$ are the signals we reconstruct with);
- if $\mathcal{T}$ is **orthonormal**, quantization error is preserved (energy is conserved);
- we then seek the $\mathcal{T}$ that **minimizes the GM** of Y (maximal energy concentration).

Paradigm:

$$x \;\to\; y = \mathcal{T}x \;\to\; \hat{y} = Q(y) \;\to\; \hat{x} = \mathcal{T}^{-1}\hat{y}$$

A coding quality is determined by the coding gain:

$$G_{\mathcal{T}} = \frac{\sigma^2_{AM,Y}}{\sigma^2_{GM,Y}}$$

Orthogonal Transform

For OT we have:

- Inverse is immediate: $\mathcal{T}^{-1} = \mathcal{T}^T$
- They are isometries (keep $\mathcal{L}^2$ norm): $\|Y\|^2 = \|\mathcal{T}X\|^2 = \|X\|^2$

Since it is an isometry, the distortion is the same:

$$MD_Y = \mathbb{E}[\left\|Y - \hat{Y}\right\|^2] = \mathbb{E}[\left\|\mathcal{T}X - \mathcal{T}\hat{X}\right\|^2] = \mathbb{E}[\left\|\mathcal{T}(X - \hat{X})\right\|^2] = \mathbb{E}[\left\|X - \hat{X}\right\|^2] = MD_X$$

And also the AM (avg of variances):

$$\sigma^2_{AM,Y} = \frac{1}{M}\sum \mathbb{E}[Y_k^2] = \frac{1}{M}\mathbb{E}[\sum Y_k^2] = \frac{1}{M}\mathbb{E}[\|Y\|^2] = \frac{1}{M}\mathbb{E}[\|X\|^2] = \sigma^2_{AM,X}$$

Where logically, the quantizers used are the ones that block code optimized for Y, so we can call this distortion the **transform distortion** $D_{\mathcal{T}}$ ($D_{\mathcal{T}} = D_X = D_Y$), while the **uniform rate (PCM)** distortion is D_{PCM}.

We define the **coding gain** as the ratio:

$$G_{\mathcal{T}} = \frac{D_{PCM}}{D_{\mathcal{T}}} = \frac{c\,\sigma^2_{AM}\,2^{-2\overline{R}}}{c\,\sigma^2_{GM,Y}\,2^{-2\overline{R}}} = \frac{\sigma^2_{AM,Y}}{\sigma^2_{GM,Y}} \geq 1$$

PCM distortion (input is id → equal variances → $\sigma^2_{GM} = \sigma^2_{AM}$ → trivial allocation, same rate $\overline{R}$):

$$D_{PCM} = c_X\sigma^2_X = c\,\sigma^2_{AM}\,2^{-2\overline{R}}$$

Transform distortion (isometry fixes σ^2_{AM}; equal shape factors on avg):

$$D_{\mathcal{T}} = c\,\sigma^2_{GM,Y}\,2^{-2\overline{R}}$$

Optimal Transform (KLT)

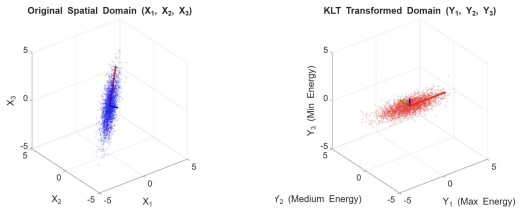
An optimal transform is the the transform that, given a random vector X with known statistical properties, has a mathematically optimal guarantee to decorellate the components and maximizes energy compaction.

The answer is the **Karhunen-Loève Transform (KLT)**
The rows of this matrix are the eigenvectors of the correlation matrix $R_X = \mathbb{E}[XX^T]$. This allows the transform to be oriented along the maximum variance direction

Intuition: In the Y domain, knowing the value of Y_1 provides zero linear information about Y_2. The redundancy present in the "diagonal" alignment is removed

Properties:

- Orthogonality: $T_{KLT}^{-1} = T_{KLT}^T$
- Decorrelating transform: $\mathbb{E}[Y_i Y_j] = \lambda_i \delta_{ij}$
- Best energy concentration (sparsity): $\sum \mathbb{E}[Y_i^2] \geq \sum \mathbb{E}[(TX)^2]$ where T is any other transform
- Optimal for gaussian RV: $\sigma^2_{GM,Y} \leq \sigma^2_{GM,TX}$



Original domain: high 3D correlation KLT domain: aligned and decorrelated

Example

Most coefficients will have near zero variance, so these can be discarded or coarsely quantized since the don't impact the MSE much.

It maximizes the coding gain as gaussians will be id:

$$G_{\mathcal{T}} = \frac{\sigma^2_{AM,Y}}{\sigma^2_{GM,Y}} = \frac{\frac{1}{M}\sum \sigma_i^2}{\left(\prod \sigma_i^2\right)^{1/M}}$$

However:

- Requires $O(n^3)$ to find eigenvectors, $O(n^2)$ for the multiplication
- Since data dependent, it must be sent to the decoder as metadata
- Model is often not stationary!

Frequency Transforms (DFT, DCT)

For a markov process with correlation $\rho \to 1$ frequency transforms offer near optimal performance (with fixed basis functions ad FFT algorithms $O(n\log n)$).

Discrete Fourier Transform (DFT)

The 1D case is the following:

$$y[k] = \frac{1}{\sqrt{M}} \sum_{n=1}^{M} x[n] e^{-j\frac{2\pi}{M}kn}$$

Or in matrix form:

$$\mathcal{T}_{DFT} = \frac{1}{\sqrt{M}} \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & W_M & W_M^2 & \dots & W_M^{M-1} \\ 1 & W_M^2 & W_M^4 & \dots & W_M^{2(M-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_M^{M-1} & W_M^{2(M-1)} & \dots & W_M^{(M-1)(M-1)} \end{bmatrix}$$

Where $W_M = e^{-j2\pi/M}$ is the M-th primitive root of unity
Each row is therefore the conjugate of a basis vector.
the total energy is preserved: $\|y\|^2 = \|x\|^2$

In 2D, the DFT is a separable transform:

$$Y = \mathcal{T} X \mathcal{T}^T$$

this computes the rows and then the columns (horizontal and vertical frequency analysis)
This transform decomposes the image into a weighted sum of N^2 orthogonal basis patterns:

$$B_{k,l}(n,m) = \frac{1}{N} e^{j\frac{2\pi}{N}(kn+lm)}$$

each coefficient $Y[k,l]$ represents the frequency component of horizontal and vertical frequency combination
Most energy is concentrated in the low frequencies.

This is still not ideal since the DFT suposes a periodic signal. We compress finite signals and therefore on the image edges we have spectral leakage (low frequencies leak into high ones) making the signal less sparse

Discrete Cosine Transform (DCT)

A general approach is used **Discrete Cosine Transform (DCT)**

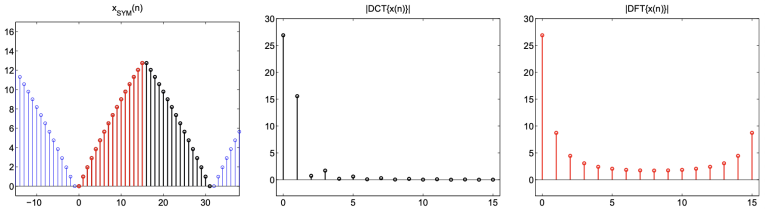
- Create mirrored-periodic version of the signal
- Compute DFT of signal+mirror
- Apply frequency domain modulation to obtain symmetrical and real valued oceffieicnts
- Keep only M coefficients (og signal length)

Which can be rewritten in a signle transform where each entry follows the form:

$$(\mathcal{T}_{DCT})_{k,n} = \begin{cases} \frac{1}{\sqrt{N}} & k = 0 \\ \sqrt{\frac{2}{N}} \cos(\frac{(2n+1)k\pi}{2N}) & k > 0 \end{cases}$$

DFT is not used since DFT has high frequency components near the signal edges. The DCT is a way to mirror the signal before the periodicity. It has only positive frequencies

Applying the DCT to a signal (a sequence of M real numbers) produces M real coefficients and has a better sparsification property than DFT thanks to the symmetric periodization.



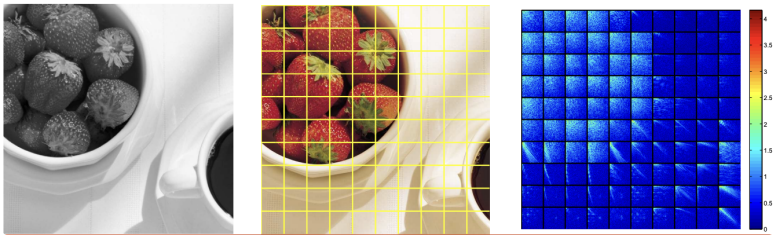
Example With Mirroring

The DCT is also separable:

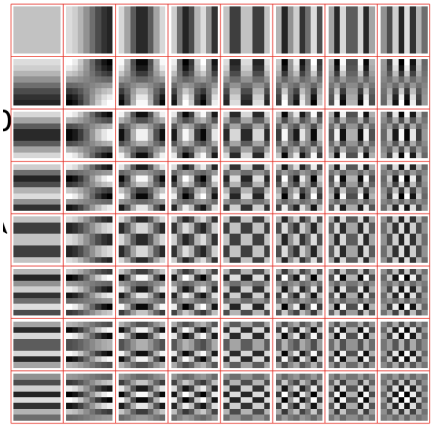
$$Y = \mathcal{T}_{DCT} X \mathcal{T}_{DCT}^T$$

A large-size, non-stationary image is more conveniently represented by dividing it into small blocks.

Example: 8 × 8 block-based DCT. Each 8 × 8 block of pixels from the image is projected onto the 64 basis vectors:
The corresponding scalar product is the DCT coefficient telling how much the block is similar to the basis vector



Block DCT



8x8 Basis

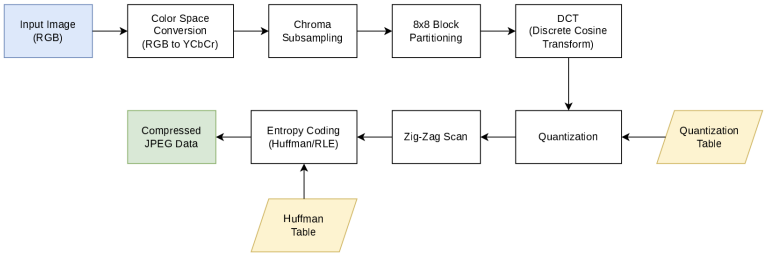
The sparsification allows to give higher bits to many small valued coefficients and lower bits to less frequent bigger values. How are the quantization coefficients computed?

- HS formula (practical implementation)
- Fixed steps (JPEG)

Large blocks: stationarity
Small blocks: correlation

4.4) JPEG standard

JPEG is an image compression standard defined in 1991 that defines **only the decoder** for interoperability and implementation competition



JPEG Scheme

This is the encoder:
Step 0; Preprocessing: includes 8×8 block creation and centering (-128 on each sample).
Step 1; DCT: 2D DCT is applied on 8×8 blocks of centered samples (find y_{ij})
Step 2; Quantization: uses a **mid tread** quantizer. The quantization table q_{ij} is not defined by the standard, therefore it must be encoded. Notice that q is the step size. A bigger q means fewer bits and therefore less quality (as an idea think of it like this $q = 255/2^R$)

$$\hat{c}_{ij} = \text{round}(\frac{y_{ij}}{q_{ij}})$$

Step 2.5; Quality Factor: The **quality factor** $Q \in [1, 100]$ that controls the scaling factor (used during quanization):

$$S_F = \begin{cases} \frac{5000}{Q} & 1 \leq Q \leq 50 \\ 200 - 2Q & 50 < Q \leq 99 \\ 1 & Q = 100 \end{cases} \rightarrow q \leftarrow \frac{S_F}{100} q$$

Step 3; Zig-Zag Scan + Entropy coding: see later

Now the decoder:
Step 0; Entropy Decode: from zigzag + entropy decode it to find quantized blocks.
Step 1; De-Quantization: multiply quantized coefficients by q.

$$\hat{y}_{ij} = \hat{c}_{ij} \cdot q_{ij}$$

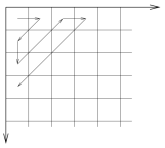
Step 2; Inverse DCT: apply the IDCT on each dequantized block and reconstruct the signal by uncentering (+128 on each sample)

Zig Zag Scan:

A zig-zag scan is performed on the quantized values in order to encode them in a single string, where

- the first value is the Difference of the DC component of this block and the previous block
- the next values are a pair of numbers representing (# of zeroes in scan, value of first non zero)
- final EOB special symbol is added to end the string it is $(0, 0)$

$DC_n - DC_{n-1}$	(# of zeroes, non zero coeff value)	...	**EOB** $(0, 0)$



Zig-Zag scan

-2	17	3	1	0	0	0	0
3	4	0	-1	0	0	0	0
0	0	-1	0	0	0	0	0
0	0	-1	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

-2-DC_{n-1}	0,17	0,3	1,4	0,3	0,1	5,-1	0,-1	4,-1	5,-1	EOB

Example

Entropy Coding of DC Coefficients

JPEG encodes the DC difference with a pseudo-huffman code.
There are $k \in \{0, \dots, 11\}$ categories, each of them holding 2^k values using 2's complement. Each category is assigned a codeword (see table).

Call DC_p the DC difference, the category is chosen $k = \lceil \log_2(|DC_P| + 1) \rceil$
To the category the binary value of DC_P is added as a suffix. If it is negative each value is complemented.

Category	Code
0	00
1	010
2	011
3	100
4	101
5	110
6	1110
7	11110
8	111110
9	1111110
10	11111110
11	111111110

Category Code

For example suppose $DC_P = -5_{10} = 101_2$ that must be complemented to 010. The category is $k = 3 \rightarrow 100$ and thus the codeword is 100 010.

DC values have a range $\in [-1024, 1060]$ a difference of two DC signals is $\in [-2040, 2040]$ and thus with cardinality 4081. Since there are 11 categories we have $\sum_{k=0}^{11} 2^k = 2^{12} = 1096 > 4081$ values.

Entropy Coding of AC Coefficients

Recall that these coefficients are previously encoded as (a, b). These values are used to find the pseudo huffman entropy encoded codeword which is prefix+:

$$[(a, k) \text{ in RC table prefix} \mid b \text{ DC pseudo huffman}]$$

That is:

- First compute $k = \lceil \log_2(|b| + 1) \rceil$ and b in binary
- Then find the prefix code corresponding to (a, k) (table)
- Put prefix + b together

Example: Encode (3,16):

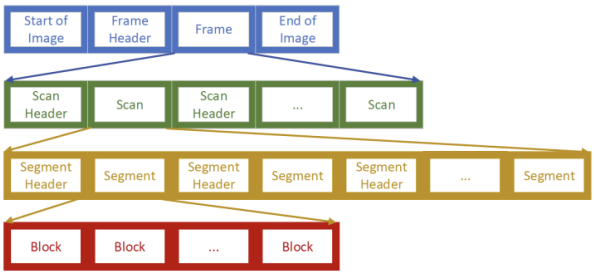
- $k = \lceil \log_2 17 \rceil = 5$ and $b = 10000_2$
- In tabe see that $(3, 5) = 1111111110010000$
  The codeword is: 1111111110010000 10000

Two custom codewords are described:

- End Of Block (EOB) $= (0, 0) \rightarrow 1010$
- Zero Run (ZR) $= (15, 0) \rightarrow 11111111001$

Frame Building

The standard frame follows this logic:



Frame

- Frame header contains static info (size, color space, digitalization format)
- Image is stored in a frame as various scans
- Scan header contains quantization table (luminance and chrominance)
- A single scan contains various segments, each segment is a concatenation of blocks
- Segment header contains huffman tables

JFIF (JPEG File Interchange Format) is the standard format for metadata in JPEG files

4.5) Chapter Recap

Block coding (resource allocation problem)
Minimize $\mathcal{D} = \frac{1}{M}\sum_k c_k \sigma_k^2 2^{-2R_k}$ subject to $\sum_k R_k = R_{tot}$.

- Optimal rate (Huang–Schultheiss):

$$R_k^* = \overline{R} + \frac{1}{2}\log_2 \frac{c_k \sigma_k^2}{c_{GM}\sigma_{GM}^2}, \qquad \overline{R} = \frac{R_{tot}}{M}$$

- Optimum **equalizes** per-sample distortion, so the global distortion depends only on the GM:

$$\mathcal{D}^* = c_{GM}\,\sigma_{GM}^2\,2^{-2\overline{R}}$$

AM vs GM. $z_{GM} \le z_{AM}$ (Jensen, log concave), equality $\iff$ all z_k equal.

Coding gain. Orthonormal transform conserves energy (σ_{AM}^2 fixed by isometry, $D_X = D_Y$) but lowers σ_{GM}^2 (energy concentration):

$$G_{\mathcal{T}} = \frac{D_{PCM}}{D_{\mathcal{T}}} = \frac{\sigma_{AM,Y}^2}{\sigma_{GM,Y}^2} \ge 1, \qquad = 1 \iff \text{all } \sigma_k^2 \text{ equal}$$

$$D_{PCM} = c\,\sigma_{AM}^2\,2^{-2\overline{R}}, \qquad D_{\mathcal{T}} = c\,\sigma_{GM,Y}^2\,2^{-2\overline{R}}$$

$\Rightarrow$ block coding helps **only with variance disparity**; the transform exists to create it.

KLT. Rows = eigenvectors of $R_X = \mathbb{E}[XX^T]$. Decorrelates ($\mathbb{E}[Y_iY_j] = \lambda_i\delta_{ij}$); optimal energy compaction (top-p partial energy maximal $\forall p$); minimizes $\sigma_{GM,Y}^2$ $\Rightarrow$ maximizes $G_{\mathcal{T}}$. For Gaussian X: decorrelation $\Rightarrow$ **independence** $\Rightarrow$ rate–distortion optimal. Cost: $O(n^3)$, data-dependent (basis must be transmitted), assumes stationarity.

DCT. Fixed basis, $O(n\log n)$, no basis to transmit; near-optimal for Markov sources with $\rho \to 1$. Symmetric periodization avoids DFT edge leakage $\to$ real, sparser coefficients.

Modified Huang–Schultheiss (fixes negative / non-integer rates)

1. Run HS on all M components.
2. Any $R_k < 0 \to$ set $R_k = 0$, drop that component, re-run HS on the rest.
3. Repeat until all rates ≥ 0; then **floor** them.

4. Distribute leftover bits ($R_{tot} - \sum \lfloor R_k \rfloor$) to the components rounded down the most.

Greedy bit allocation (same result, $O(R_{tot})$)

- Init: $R_k = 0$, $D_k = \sigma_k^2 \; \forall k$.
- Repeat R_{tot} times:
  - $l = \arg\max_k D_k$
  - $R_l \mathrel{+}= 1$, then $D_l \mathrel{/}= 4$ (since one extra bit quarters distortion).

KLT construction

1. Estimate $R_X = \mathbb{E}[XX^T]$.
2. Eigendecompose: $R_X = U \Lambda U^T$.
3. Transform $Y = U^T X$ (rows of $\mathcal{T}_{KLT} = U^T$ are the eigenvectors).
4. Coefficient Y_i has variance λ_i; keep / finely quantize large λ_i, discard small ones.

JPEG encoder
0. **Preprocess:** 8×8 blocks, level-shift -128.

1. **DCT:** 2-D DCT per block → y_{ij}.
2. **Quantize:** build table (base × quality Q, clamp $\max(1, \cdot)$); mid-tread round $\ell_{ij} = \text{round}(y_{ij}/q_{ij})$ → small coeffs become 0.
3. **Zig-zag + RLE:** DC differential ($DC_n - DC_{n-1}$); AC as (run-of-zeros, value); EOB $(0, 0)$, ZRL $(15, 0)$.
4. **Entropy code:** pseudo-Huffman (category $k = \lceil \log_2(|v| + 1) \rceil$ + magnitude bits) → bitstream. Quant + Huffman tables go in the header.

JPEG decoder (reverse)
0. **Entropy decode** bitstream → levels ℓ_{ij}.

1. **De-quantize:** $\hat{y}_{ij} = \ell_{ij} \, q_{ij}$.
2. **IDCT** per block.
3. **Uncenter** $+128$, clamp to $[0, 255]$.

Block coding defines *what a good transform must do* (maximize $\sigma^2_{AM}/\sigma^2_{GM}$) → KLT does it **optimally** → DCT does it **cheaply** → JPEG **packages** it. Transform creates redundancy as variance disparity, allocation spends bits on it, entropy coding harvests the resulting zeros.

5) Wavelet Transform

Recall the principle of a spectrum analyzer (short time fourier transform) (DSP 2). Frequency and time resolutions are inversely proportional to each other

$$\text{Heisenberg-like Uncertainty principle: } \Delta t \cdot \Delta f \geq \frac{1}{4\pi}$$

Wavelet is the tool that allows the block of the JPEG to scale dynamically based on frequency (high frequency, smaller blocks. Low frequency, large blocks). In fact an image is made of two parts:

- **Anomalies:** High frequency content (edges, contours). This needs a good time resolution to see where they are located
- **Trends:** low frequency content (smooth areas, textures). This needs good frequency resolution to better capture subtle shifts in the image

To achieve this we use a **mother wavelet** $\psi(t)$ and generate the basis through scaling and translation:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}}\psi\left(\frac{t-b}{a}\right)$$

This works with the

Theorem 9 (Universal Principle)**.**

The linear transforms used in signal processing and compression are defined by projection of the input signal onto an appropriate set of basis functions.

Given an orthonormal basis $\{\phi_k(t)\}$ *any signal can be perfectly represented as*

$$x(t) = \sum_k c_k \phi_k(t)$$

and the coefficient is obtained by

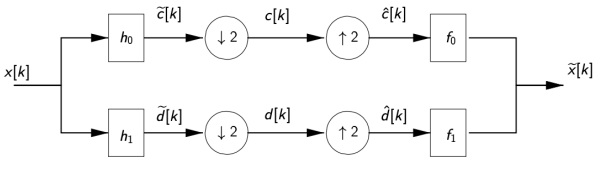
$$c_k =< x(t), \phi_k(t) >= \int x(t)\phi_k^*(t)dt$$

5.1) Discrete Wavelet Transform

Discrete Wavelet Transform starts with the filter bank, collections of filter that divide the signal in different subbands:

Filter Bank

The idea is to divide the signal in two parts: high and low frequency. These will have their bandwidth halved and so they get decimated and interpolated with a factor of 2. These get recombined to get a delayed copy of the original signal.



Filter Bank Scheme

$$x[n] \xrightarrow{H_i} \tilde{c}_i[n] \xrightarrow{\downarrow 2} c_i[k] \xrightarrow{\uparrow 2} \hat{c}_i[n] \xrightarrow{F_i} v_i[n] \xrightarrow{\sum} \tilde{x}[n]$$

where these steps are followed:

$$\begin{aligned}
H_i \text{ is the bandpass filter: } \tilde{c}_i[n] &= (h_i * x)[n] & \tilde{C}_i(z) &= H_i(z)X(z) \\
\text{Downsampling is done as: } c_i[n] &= \tilde{c}_i[2n] & C_i(z) &= \frac{1}{2}[\tilde{C}_i(z^{1/2}) + \tilde{C}_i(-z^{1/2})] \\
\text{Upsampling isdone as: } \hat{c}_i[n] &= \begin{cases} c_i[n/2] & n \text{ even} \\ 0 & n \text{ odd} \end{cases} & \hat{C}_i(z) &= C_i(z^2) \\
F_i \text{ is the synthesiys filter: } v_i[n] &= (f_i * \hat{c}_i)[n] & V_i(z) &= F_i(Z)\hat{C}_i(z) \\
\text{Final output: } \tilde{x}[n] &= \sum v_i[n] & \tilde{X}(z) &= \sum V_i
\end{aligned}$$

Filter banks have these properties:

- **Perect Reconstruction (PR):** see later
- **Invertible Modulation Matrix:** see later
- **Finite Impulse Response**
- **Orthogonality**
- **Vanishing Moments**
- **Symmetric**

Only 2 filters have these properties:

- **Quadrature Mirror Filters (QMF):**

$$H_0(z) = H_1(-z) \text{ and } F_0(z) = H_0(z),\ F_1(z) = -H_1(z)$$

- **Conjugate Quadrature Filters (CQF):**

$$H_0(z) = H_1(-z) \text{ and } F_0(z) = H_0(z^{-1}),\ F_1(z) = -H_1(z^{-1})$$

  Both are Orthogonal and energy conserving. A special case is the **Haar filter**, which is both A QMF and CQF at the same time (see later).

Perfect Reconstruction

Let $x[k]$ be the original signal and $\tilde{x}[k]$ the signal after passing through the filter bank. **Perfect reconstruction** is achieved if $\tilde{x}[k]$ is a delayed copy of $x[k]$, that is:

$$\tilde{x}_k = x_{k+l} \iff \tilde{X}(z) = z^{-l}X(z)$$

Perfect reconstruction analysis of a 2-channel filter bank is:

$$\tilde{X}(z) = \frac{1}{2}\underbrace{[F_0H_0(z) + F_1H_1(z)]}_{T(z)}X(z) \ + \ \frac{1}{2}\underbrace{[F_0H_0(-z) + F_1H_1(-z)]}_{A(z)}X(-z)$$

By recalling the definition of perfect reconstruction we can build a system by defining:

- Non distortion (ND) conditions: $T(z) = 2z^{-l}$
- Aliasing Cancellation (AC): $A(z) = 0$

$$\tilde{X}(z) = \frac{1}{2}\underbrace{\begin{bmatrix} H_0(z) & H_1(z) \\ H_0(-z) & H_1(-z) \end{bmatrix}}_{\text{modulation matrix}} \cdot \begin{bmatrix} F_0(z) \\ F_1(z) \end{bmatrix} X(z) = \frac{1}{2}\begin{bmatrix} 2z^{-l} \\ 0 \end{bmatrix} X(z) = z^{-l}X(z)$$

Proof of reconstruction:

$$
\begin{aligned}
\tilde{X} &= v_0 + v_1 \\
&= F_0\hat{C}_0 + F_1\hat{C}_1 \\
&= F_0C_0(z^2) + F_1C_1(z^2) \\
&= \frac{1}{2}F_0[\tilde{C}_0(z^{1/2}) + \tilde{C}_0(-z^{1/2})] + \frac{1}{2}F_1[\tilde{C}_1(z^{1/2}) + \tilde{C}_1(-z^{1/2})] \\
&= \frac{1}{2}F_0(H_0(z)X(z) + H_0(-z)X(-z)) + \frac{1}{2}F_1(H_1(z)X(z) + H_1(-z)X(-z)) \\
&= \frac{1}{2}\left[F_0H_0(z) + F_1H_1(z)\right]X(z) \; + \; \frac{1}{2}\left[F_0H_0(-z) + F_1H_1(-z)\right]X(-z)
\end{aligned}
$$

Modulation Matrix Invertibility

The modulation matrix needs to be invertible:

$$
\tilde{X}(z) = \frac{1}{2}\underbrace{\begin{bmatrix} H_0(z) & H_1(z) \\ H_0(-z) & H_1(-z) \end{bmatrix}}_{\text{modulation matrix}} \cdot \begin{bmatrix} F_0(z) \\ F_1(z) \end{bmatrix} X(z)
$$

$$
\forall z \in \mathbb{C} : |z| = 1, \; \Delta(z) = H_0(z)H_1(-z) - H_1(z)H_0(-z) \neq 0
$$

Orthogonality

QMF and CQF are orthonogal to **ensure energy conservation:**

$$
\sum (x_k)^2 = \sum (c_k)^2 + \sum (d_k)^2
$$

Vanishing Moments

VM represents the ability of a filter to reproduce polynomials:
A filter with VM of p:

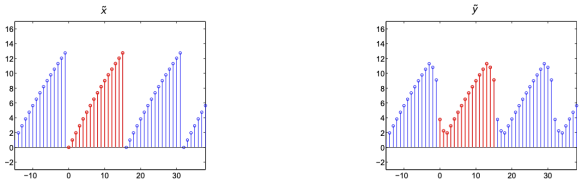
- Can represnet polynomial of degree up to p
- Has $2p$ taps

The HP filter looses infomation as it has a finite tap, but it's information remains in

Border problems

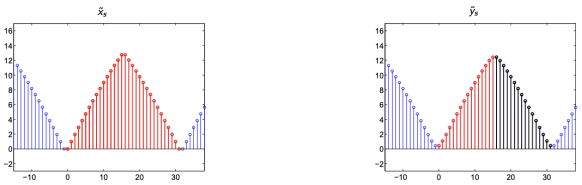
What we have seen works for 1D infinite signal, but images are 2D (no problem since filters are separable) and with **finite support.** We have 3 approaches:

- **Standard approach:** zero padding and DTFT. This produces **coefficient expansion,** the output signal has $N + M - 1$ coefficients (N input size and M filter size).
- **Circular Convolution:** This is obtained via periodicization of the signal. This however introduces boundary artifacts (aliasing in frequency) because of the implicit periodicization of the circular convolution (DFT)



Example Of Boundary Artifacts

- **Symmetrization:** We create a new signal by adding a mirror image of the original signal to the period. Let x have period N then x_s has period $2N$. The circular convolution (with periodic filter) will return a periodic and symmetrical signal and thus only the first N samples have to be computed. This does not create artifacts.



Example of Symmetric signal (in black the N not calculated coefficients)

However this doubles the filter coefficients unless the filter is symmetric!

Haar and Biorthogonal Filters

The only symmetric FIR orthogonal filter is the Haar filter

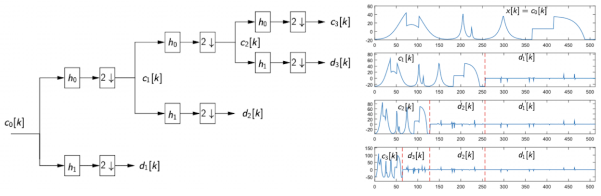
$$h_0[k] = [1,1] \quad f_0[k] = [1,1]$$
$$h_1[k] = [1,-1] \quad f_1[k] = [-1,1]$$

Unfortunately this is a filter with Vanishing moment (VM) of $p = 1$. The high pass filter will not respond to polynomials with degree $< p$. We need at least $2p$ taps.

Haar has $p = 1$ and can only represent piecewise linear functions.
We consider the **Cohen-Daubechies-Fauveau (CDF)** biorthogonal filter:

- Symmetric
- Maximize VM
- close to orthogonal ($\omega_i \approx 1$)

Here is a 3 level wavelet decomposition

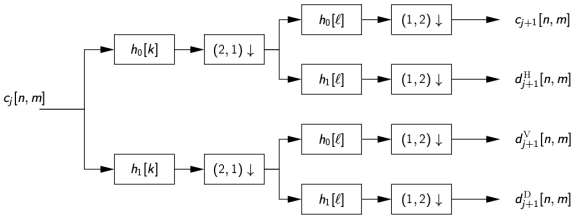


Three wavelet decomposition

and then reconstructing the signal in order.

2D Wavelet Decomposition

For 2D signals it is possible to use this schema:



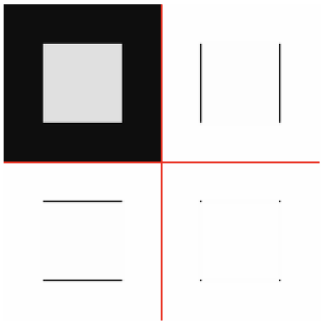
2D

and for multi level decomposition just the output c_i is used.
The 4 outpust are:

- c (A): approximation coefficients (low res version of og image by LP filter in both directions)
- d^H (H): horizontal HP (HP on rows, LP on cols)
- d^V (V): vertical HP (HP on cols, LP on rows)

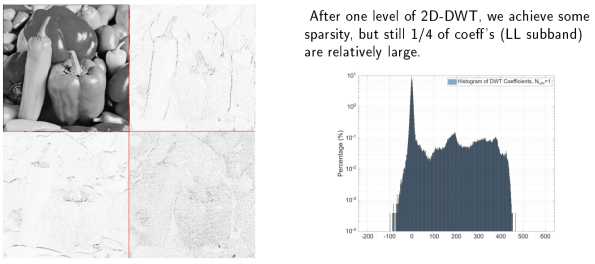
- d^D: diagonal details

Here you can see the various outputs after one level of decomposition

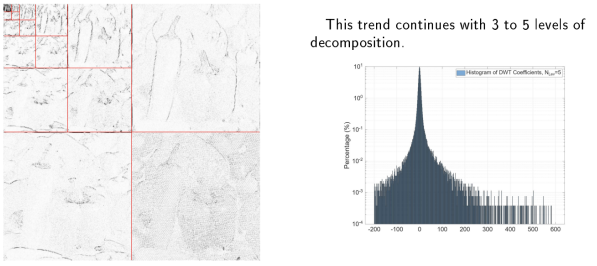


Example

Applying this sparsifies the signal:



One Level



5 Levels

the optimal levels are between 4-6.

5.2) Image Compression with Wavelets

We will study two approaches:

- Inter Scale Dependency (EZW): Tree based representation, low complexity with inter scale dependency, not resolution scalable
- Inter Scale Dependency (JPEG 200): Explicit bit rate allocation + entropy coding. Reslution scalable and allows for random access, no intra scale dependency.

Embedded Zerotrees of Wavelet Coefficients (EZW)

Here are the properties:

- Quality scalable: progressive representation
- Lossles to lossy
- Small complexity
- RD better than JPEG

Exploits **auto similarity:** when a coeff is small, also its descendants are, then we can save with just 1 symbol

Construction of Auto Similarity Principle:

Each new bit should convey max information: send first big coefficients, but has **localization overhead**: subband scan+ zero tree

One (partial solution) is the **subband scan:** scan in order C,H,V,D from smallest to highest subband

We also need a way to find biggest coeff first without sending localization info. Exploit inter-band correlation to predict position of non significant coeff.
One pixel in the subband has 4 times as more coeff in the next band. **If a coeff is small, also its descendants are**.
A (sub-)tree of below-the-threshold coefficients is called a zero-tree. it is encoded with only one symbol.

EZW Algorithm

This allows to encode the bitplane $\log_2 T_k$ at the k-th pass where each new bitplae refines the coeff quantization. Significant symbols are losslessly encoded

Heuristic:
Walk the tree coarse-to-fine. If you find a big coefficient, announce it (SP/SN) and move it to SS S. If you find a small coefficient whose whole subtree is also small, prune it (ZR). If you find a small coefficient with a significant descendant somewhere below, mark it IZ and keep scanning. Then halve the threshold and do it again, but first refine everything already in SS S by one more bit.

It is done in these steps:

1. Set $k = 0$, $n = \lfloor \log_2 |c_{\max}| \rfloor$, $T_k = 2^n$
2. Let $\mathcal{L}$ be the list of coeff in SB scan.Let $S = 0$ be the list of significant coeff
3. while (rate<available rate)
    1. Dominant pass
    2. Refining pass
    3. $T_{k+1} \leftarrow T_k/2$
    4. $k \leftarrow k + 1$
4. end

Dominant pass:

1. Until $\mathcal{L}$ is not empty:
    1. Let c be the first coeff in the list
    2. If $c > T_k$ encode c as Significant Positive (SP) and put in S
    3. elif $c < -T_k$ ancode as Significant Negative (SN) and put in S
    4. elif no desc bigger than T_k (in abs), then
        1. Encode c as ZR (zero tree root)
        2. remove desc from $\mathcal{L}$
    5. Else encode c as isolated zero (IZ)
2. Remove c from $\mathcal{L}$

Refining Pass:

1. Let $b = \log : 2T_k$ the currwnt bit plane index
2. For all c in S encothe the b-th bit of binary representation

Example (SEE BETTER)

Consider this image:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Example

- **Bitplane 1:**
  Notice $T = 2^4$
  Dominant pass
  Since $26 \geq 16$ it is SP
  Then $6 \leq 16$ and also its descendants $(-12, 10, 6, 4)$ so ZR
  Same for $-7, 3$
  At the end of dominant pass at bitlplane 1 we have:

  $$\text{SP,ZR,ZR,ZR}$$

  Refinement Pass
  One SP, find it's $\log_2 T = 4$ bit: $26_{10} = 11010_2 \rightarrow$ select 1

  $$\text{SP,ZR,ZR,ZR,1}$$

- **Bitplane 2:**
  Notice $T = 2^3$
  Dominant Pass:
  Since $6 < 8$ but has descendant $|-13| > 8$ it is IZ
  Next $-7, 3$ are ZR
- Now scan subband 2:
  $|-13| \geq 8$ set SN
  $10 \geq 8$ SP
  Next two $6, 4$ are ZR
  Dominant pass sets

  $$\text{IZ,ZR,ZR,SN,SP,ZR,ZR}$$

  Refinement Pass
  We have 3 significant coeff $n = 3$: $26 = 11010 \rightarrow 0$ $13 = 1101 \rightarrow 1$ $10 = 1010 \rightarrow 0$

  $$\text{IZ,ZR,ZR,SN,SP,ZR,ZR,}0, 1, 0$$

- **Bitplane 3**
  Notice $T = 2^2$
  Dominant pass:
  $6 \geq 4$ SP
  $-7 \leq -4$ SN
  $3 < 4$ ZR
- Level 2 scan
  $6 \geq 4$ SP
  $4 \geq 4$ SP
  $4 \geq 4$ SP
  The remaining 3 $-3, 2, -2$ are ZR
  Dominant pass sets

  $$\text{SP,SN,ZR,SP,SP,SP,ZR,ZR,ZR}$$

  Refinement Pass:
  We have 8 significant coeff $(26, 13, 10, 6, 7, 6, 4, 4)$

  $$\text{SP,SN,ZR,SP,SP,SP,ZR,ZR,ZR,}1, 0, 1, 1, 1, 1, 0, 0$$

  Final bitstream:

  $$\text{SP,ZR,ZR,ZR,}1, |\text{IZ,ZR,ZR,SN,SP,ZR,ZR,}0, 1, 0, |\text{SP,SN,ZR,SP,SP,SP,ZR,ZR,ZR,}1, 0, 1, 1, 1, 1, 0, 0$$

Decoding:
Initial state all coeff unknown
Receive SP at $[1,1]$, we know $1xxxx$ so we choose midway $11000 = 24$
Then all ZR so see ref pass: get bit 1, choose $11xxx \to 11100 = 28$

Receive SN at $[1,3]$ we know $-01xxx$ choose $-01100 = -12$
The SP at $[1,4]$ we know $01xxx \to 01100 = 12$
Then all ZR so see ref pass: get 0,1,0 so we know:

$$110 \to 11010 = 26, \ -011xx \to -01110 = -14, \ 010xx \to 01010 = 10$$

For SP,SN,ZR at level 1 we have $\pm 001xx \to \pm 00110 = \pm 6$ and ZR is 0
For the level 2 SP we have 6 again
Then only ZR so see ref pass:

$$1101x \to 27, \ -0110x \to -13, \ 0101x \to 11$$

Now the SP,SN at level 1:

$$0011x \to 7, 1 - 0011x \to -7$$

At level 2:

$$0011x \to 7, \ 0010x \to 5, \ 0010x \to 5$$

so we have

$$27, 7, -7, 0| -13, 11, 7, 5|5, 0, 0, 0|0, 0, 0, 0$$

Decoded:

27	7	-13	11
-7	0	7	5
5	0	0	0
0	0	0	0

True:

26	6	-13	10
-7	3	6	4
4	-3	3	-3
2	-2	-2	0

Final result

JPEG 2000

Here are the properties:

- ROI coding
- Quality and resolution scalable
- Tiling
- Exact coding rate
- Lossless to lossy

Algorithm

Made of 2 tiers:

1. DWT + quantization and lossless coding of codeblocks
2. EmBedded Block Coding with optimized Truncation (EBCOT) and Scalability + ROI

DWT is encoded with fine quantization steps
For lossless coding, DWT are ints and are not quantized
No loss in DWT, we have loss in tier 2

EBCOT

Each subband split in equally sized blocks (codeblocks) losslessly and independently coded via arithmetic: we get as many bitstreams as blocks

Example

These get truncated:

$$\min \sum D_i \qquad \sum R_i \leq R_{tot}$$

Solution:
Optimal truncation when slope of $D_i(R_i)$ are equal

5.3) Error Robustness

One single bit error can introduce to many wrong decoding steps

- Correction code: increase rate used only on small sensitive data
- Markers: increased rate insert into bitstream to stop propagation errors

JPEG200 has implicit markers (codeblock independently coded)

5.4) Recap

Recall the STFT / spectrum analyzer (DSP 2): time and frequency resolution are inversely proportional.

$$\text{Heisenberg-like:} \quad \Delta t \cdot \Delta f \geq \frac{1}{4\pi}$$

Wavelets make the analysis window **scale with frequency**, so the JPEG-style block adapts:

- **Anomalies** = HF (edges, contours): want fine **time** resolution → short window.
- **Trends** = LF (smooth areas, textures): want fine **frequency** resolution → long window.
  Mother wavelet, generated by scaling + translation:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}}, \psi\left(\frac{t-b}{a}\right)$$

> **Theorem 10** (Universal Principle Linear transforms in signal processing/compression = projection of the signal onto a chosen basis. For an orthonormal basis $\{\phi_k(t)\}$:
>
> $$x(t) = \sum_k c_k, \phi_k(t), \qquad c_k = \langle x, \phi_k \rangle = \int x(t), \phi_k^*(t), dt$$
>
>).

5.5) Discrete Wavelet Transform

DWT = recursive **filter bank**: split into HF/LF subbands, each band has half the bandwidth ⇒ decimate ↓2, then ↑2 + synth, recombine into a delayed copy. Recurse on the **LP branch only**.

Filter-bank pipeline:

$$x[n] \xrightarrow{H_i} \tilde{c}_i[n] \xrightarrow{\downarrow 2} c_i[k] \xrightarrow{\uparrow 2} \hat{c}_i[n] \xrightarrow{F_i} v_i[n] \xrightarrow{\sum} \tilde{x}[n]$$

Step	n-domain	z-domain
Analysis filter H_i	$\tilde{c}_i[n] = (h_i * x)[n]$	$\tilde{C}_i(z) = H_i(z)X(z)$
Downsample $\downarrow 2$	$c_i[k] = \tilde{c}_i[2k]$	$C_i(z) = \frac{1}{2}\left[\tilde{C}_i(z^{1/2}) + \tilde{C}_i(-z^{1/2})\right]$
Upsample $\uparrow 2$	$\hat{c}_i[n] = \{c_i[n/2] \quad n \text{ even } 0 \quad n \text{ odd}$	$\hat{C}_i(z) = C_i(z^2)$
Synthesis filter F_i	$v_i[n] = (f_i * \hat{c}_i)[n]$	$V_i(z) = F_i(z)\hat{C}_i(z)$
Output	$\tilde{x}[n] = \sum_i v_i[n]$	$\tilde{X}(z) = \sum_i V_i(z)$

The $X(-z)$ term born at $\downarrow 2$ is **aliasing**; it survives $\uparrow 2$ and must be cancelled by the synthesis bank.

Perfect Reconstruction

Definition 11 (Perfect Reconstruction $\tilde{x}[k]$ is a pure delayed copy of $x[k]$:

$$\tilde{X}(z) = z^{-l}X(z) \iff \tilde{x}[k] = x[k-l]$$

).

2-channel analysis (collect the $X(z)$ and $X(-z)$ terms):

$$\tilde{X}(z) = \underbrace{\tfrac{1}{2}\left[F_0H_0(z) + F_1H_1(z)\right]}_{T(z)}X(z); + ; \underbrace{\tfrac{1}{2}\left[F_0H_0(-z) + F_1H_1(-z)\right]}_{A(z)}X(-z)$$

Two conditions:

- **Non-distortion (ND):** $T(z) = z^{-l} \iff F_0H_0(z) + F_1H_1(z) = 2z^{-l}$
- **Aliasing cancellation (AC):** $A(z) = 0 \iff F_0H_0(-z) + F_1H_1(-z) = 0$

Matrix form, **modulation matrix**:

$$\tilde{X}(z) = \tfrac{1}{2}\underbrace{\begin{bmatrix} H_0(z) & H_1(z)H_0(-z) & H_1(-z) \end{bmatrix}}_{\text{modulation matrix}}\begin{bmatrix} F_0(z) \\ F_1(z) \end{bmatrix}X(z) = \tfrac{1}{2}\begin{bmatrix} 2z^{-l} & 0 \end{bmatrix}X(z) = z^{-l}X(z)$$

Invertibility: solvable for (F_0, F_1) iff the determinant is non-zero on the unit circle:

$$\forall z,\ |z| = 1: \quad \Delta(z) = H_0(z)H_1(-z) - H_1(z)H_0(-z) \neq 0$$

Proof (chain-substitute the steps):

$$\tilde{X} = F_0\hat{C}_0 + F_1\hat{C}_1 = F_0C_0(z^2) + F_1C_1(z^2) \& = \tfrac{1}{2}F_0\left[H_0(z)X(z) + H_0(-z)X(-z)\right] + \tfrac{1}{2}F_1\left[H_1(z)X(z) + H_1(-z)X(-\right.$$

Key move: substitute $C_i(z)$ from the $\downarrow 2$ step, then evaluate at z^2 (so $z^{1/2} \to z$). Do **not** re-apply the $\downarrow 2$ formula to $C_i(z^2)$.

QMF vs CQF

The two FIR designs that hit the PR conditions:

Analysis relation	Synthesis	Property	
QMF	$H_1(z) = H_0(-z)$ (modulation only)	$F_0 = H_0(z),\ F_1 = -H_1(z) = -H_0(-z)$	cancels aliasing, **but no PR** for FIR (len > 2): residual amplitude distortion $T(z) = \frac{1}{2}[H_0^2(z) - H_0^2(-z)]$
CQF	$H_1(z) = z^{-(N-1)}H_0(-z^{-1})$ (mod + time-reversal)	$F_0(z) = z^{-(N-1)}H_0(z^{-1}),\ F_1(z) = z^{-(N-1)}H_1(z^{-1})$	**orthogonal, energy-conserving, full PR** (Daubechies family)

That is: CQF = QMF **plus** the conjugate/time-reversal $h_1[n] = (-1)^n h_0[N-1-n]$, which buys the extra DOF for exact PR. **Haar** is the special case that is simultaneously QMF and CQF (the unique symmetric FIR orthogonal filter).

Orthogonality ⇒ energy conservation:

$$\sum_k x_k^2 = \sum_k c_k^2 + \sum_k d_k^2 \qquad \text{(Parseval split into LP } c + \text{HP } d)$$

Vanishing moments (VM):

A filter with p VM:

- HP filter **annihilates polynomials of degree** $< p$ (reproduces up to degree $p-1$).
- Needs $\geq 2p$ taps (orthogonal Daubechies: exactly $2p$).
- More VM ⇒ smoother wavelet ⇒ HP coeffs ≈ 0 on smooth regions ⇒ sparser representation.

Border problems

Theory is for 1D infinite signals; images are 2D (fine — filters are **separable**) and **finite-support**:

Approach	Idea	Issue
Standard	zero-pad + DTFT	**coefficient expansion**: output has $N + M - 1$ coeffs (N input, M filter)
Circular conv	periodicize (via DFT)	no expansion, but **boundary artifacts** (periodization discontinuity = freq aliasing)
Symmetrization	mirror-extend, period $2N$	output periodic & symmetric ⇒ compute only first N; **no artifacts**, but **doubles taps unless the filter is symmetric**

⇒ symmetrization wants **symmetric (linear-phase)** filters ⇒ motivates **biorthogonal**.

Haar & Biorthogonal (CDF)

Haar (unique symmetric FIR orthogonal filter):

$$h_0[k] = [1,1] \quad f_0[k] = [1,1] \qquad h_1[k] = [1,-1] \quad f_1[k] = [-1,1]$$

- Usually normalized by $1/\sqrt{2}$ for unit-gain exact PR (unnormalized ⇒ $\tilde{X} = 2z^{-1}X$).
- $p = 1$: HP kills only **constants** ⇒ **piecewise-constant** approximation. Poor for smooth content.

Cohen–Daubechies–Fauveau (CDF) biorthogonal:

- Symmetric (linear phase) ⇒ symmetrization with no tap doubling.
- Maximizes VM.

- Near-orthogonal ($\omega_i \approx 1$). Used by JPEG 2000.

2D Wavelet Decomposition

Separable: filter rows, then columns. One level ⇒ **4 subbands**:

Subband	Filtering	Content
c (A, LL)	LP rows, LP cols	approximation (low-res image)
d^H (H)	HP rows, LP cols	horizontal detail
d^V (V)	HP cols, LP rows	vertical detail
d^D (D)	HP rows, HP cols	diagonal detail

Multilevel: recurse on c (the LL band) only. Sparsifies the signal; **optimal 4–6 levels**.

5.5) Image Compression with Wavelets

Both paradigms run the DWT first; they differ in **how the coefficients are coded**:

EZW / SPIHT	JPEG 2000 (EBCOT)	
Exploits	**inter-scale** (zerotrees across scales)	**intra-scale** (codeblocks within a subband)
Coding	tree symbols	explicit bit-rate allocation + arithmetic
Pros	good inter-scale exploitation, low complexity	resolution + quality scalable, random access, ROI
Cons	**no resolution scalability**	**no inter-scale exploitation**

Embedded Zerotrees of Wavelet coefficients (EZW)

Properties: **quality-scalable** (progressive/embedded bitstream), lossless→lossy, low complexity, RD better than JPEG.

Auto-similarity principle: if a coeff is insignificant, its descendants likely are too ⇒ encode the whole subtree with **one symbol** (zerotree). Reduce localization overhead with:

- **Subband scan:** order C, H, V, D, coarse→fine, so big coeffs go first.
- **Inter-band prediction:** one coeff has 4 children in the next finer band; predict insignificant positions via the tree.

Four symbols:

- **SP / SN** — significant positive/negative ($|c| \geq T_k$); move to significance list S.
- **ZR** (zerotree root) — insignificant **and** all descendants insignificant ⇒ prune the subtree.
- **IZ** (isolated zero) — insignificant **but** ≥ 1 descendant is significant ⇒ cannot prune.

> **Definition 12** (IZ vs ZR Both have $|c| < T_k$. The difference is the subtree: all-small ⇒ **ZR** (prune); at least one descendant $\geq T_k$ ⇒ **IZ** (keep scanning children).)**.**

Algorithm:

$$\textbf{Init: } k=0, \quad n=\lfloor \log_2 |c|_{\max} \rfloor, \quad T_0 = 2^n$$

$\mathcal{L}$ = coeffs in subband-scan order; $S = \varnothing$.

```
while (rate < budget):
    1. Dominant pass        # significance map at T_k
    2. Refinement pass      # 1 extra bit for coeffs already in S
    3. T_{k+1} = T_k / 2
    4. k = k + 1
```

Dominant pass — for each c in $\mathcal{L}$:

1. $|c| \geq T_k,\ c > 0 \rightarrow$ **SP**, move to S
2. $|c| \geq T_k,\ c < 0 \rightarrow$ **SN**, move to S
3. $|c| < T_k$, no descendant $\geq T_k \rightarrow$ **ZTR**, remove descendants from $\mathcal{L}$
4. $|c| < T_k$, some descendant $\geq T_k \rightarrow$ **IZ**

(Newly significant coeffs leave $\mathcal{L}$; ZTR descendants are skipped this pass.)

Refinement pass — bit-plane $b = \log_2 T_k$: for every coeff already in S (from *previous* passes), output its b-th bit ⇒ one more bit of precision.

Embedded ⇒ decoder can stop anywhere. Reconstruction is **midpoint**: SP/SN at threshold T ⇒ place at $\pm 1.5T$, then each refinement bit narrows the interval.

JPEG 2000

Properties: **ROI** coding, quality **and** resolution scalable, tiling, **exact target rate**, lossless→lossy.

Two tiers:

1. **DWT + lossless coefficient coding.** Reversible integer DWT (lossless) or float DWT + fine quantization (lossy); per-codeblock lossless arithmetic coding. **No loss in the DWT itself.**
2. **EBCOT** (Embedded Block Coding with Optimized Truncation) + scalability/ROI. **Loss happens here**, by truncating block bitstreams.

EBCOT:

Each subband → equal-size **codeblocks**, each **independently** arithmetic-coded ⇒ one embedded bitstream per block. Allocate rate by RD-optimal truncation:

$$\min \sum_i D_i \qquad \text{s.t.} \qquad \sum_i R_i \leq R_{\text{tot}}$$

Optimal truncation: cut each block where the RD-curve slopes are equal,

$$\left| \frac{dD_i}{dR_i} \right| = \text{const} \quad \forall i \qquad \text{(Lagrangian / constant-slope condition)}$$

Independent codeblocks ⇒ random access + implicit resync (error containment).

Error Robustness

A single bit error in a variable-length / embedded stream can corrupt **many** subsequent decoding steps. Mitigations:

- **Error-correcting codes:** extra rate, applied **selectively** to small sensitive data (headers).
- **Resync markers:** extra rate inserted into the bitstream to **stop error propagation**.

- **JPEG 2000:** independently-coded codeblocks act as **implicit markers** — an error is contained to its block.

6) Learned Image Coding (TODO)

7) Motion Estimation

Videos are different from images as they implement temporal information. This information is mostly found in the movement. The study of **optical flow** consists in defining the movement of a pixel between two subsequent images into a **vector field**.

Optical flow consists in finding a 2D vector field $V(x, y)$:

$$V : (x, y) \in \mathscr{I} \subset \mathbb{R}^2 \to (u, v)$$

where:

- x, y are the points on the image
- $u(x, y), v(x, y)$ is the velocity of the point x, y

The output is either dense or sparse, depends on implementation.

Hypothesis 13 (Constant Illumination)**.**

The Constant Illumination Hypothesis (CIH) states that the luminance does not change along the motion trajectory:

$$f(x, y, t + T) = f(x - c, y - d, t) \longrightarrow \frac{df}{dt} = 0$$

But in practice due to sampling, aliasing and noise this is not true.

The velocity field (optical flow) can be described as

Definition 14 (Velocity Field)**.**

We can define the vector field as:

$$V(x, y) = \lim_{T \to 0} \frac{D(x, y)}{T} = \begin{bmatrix} u(x, y) \\ v(x, y) \end{bmatrix}$$

Which becomes (first degree approx)

$$u f_x + v f_y + f_t = 0$$

with u, v components of the velocity field, f_x, f_y the space derivatives and f_t the time derivative.
This formula states that, the intensity change I see in a point depends only on the movement of the pixels.

However the OF equation has 2 unknowns. We need an additional constraint to solve the problem. Also CIH isn't true in practice.

Proof:
Apply Taylor:

$$f(p, t + T) = f(p, t) - c(p) f_x(p, t) - d(p) f_y(p, t) + o(\|D(p)\|) = f(p, t) - D \nabla f + o(\|D(p)\|)$$

And now find the partial time derivative:

$$f_t \overset{T\to 0}{=} \frac{f(p,t+T)-f(p,t)}{T} = \frac{-D\cdot\nabla f}{T} + \frac{o(\|D(p)\|)}{T} = -V\nabla f + \frac{o(\|D(p)\|)}{T}$$

From here the OF equation becomes:

$$f_t = -V\nabla f \to V\nabla f + f_t = 0 \to uf_x + vf_y + f_t = 0$$

□

7.1) Variational Method (Horn Shunk)

Variational methods apply the CIH directly.
The typical formulation is minimization of the coherence D optical flow (u,v) wrt to two images f_1, f_2 with a regularization term R (a priori knowledge)

$$\arg\min_{u,v} D(f_1, f_2, (u,v)) + R(u,v)$$

Consider a point of an object moving from pixel $p-D$ to p in time T. The trajectory of the pixel becomes:

$$\begin{matrix} x(t_0) = p - D \\ x(t_0+T) = p \end{matrix} \longrightarrow D(p,t_0,T) = x(t_0+T) - x(t_0) = \begin{bmatrix} c(x,y) \\ d(x,y) \end{bmatrix}$$

where c, d depend on p, t_0, T but the time parameters are ignored and $p = (x,y)$.

We can identify a general solution:

Solution: Minimize the energy of OF equation under suitable constraints.

Horn and Schunk introduced a constrain on the total variation of V over a region $\mathscr{R}$:

$$\begin{cases} \int\int_{\mathscr{R}} (uf_x + vf_y + f_t)^2 dxdy = \min \\ \int\int_{\mathscr{R}} \|\nabla u\|^2 + \|\nabla v\|^2 dxdy \le \tau \end{cases} \to \begin{cases} u = \overline{u} - f_x \frac{\overline{u}f_x + \overline{v}f_y + f_t}{\lambda \|\nabla f\|^2} \\ v = \overline{v} - f_y \frac{\overline{u}f_x + \overline{v}f_y + f_t}{\lambda \|\nabla f\|^2} \end{cases}$$

where $\overline{z}$ is the average of local values

This gives a **dense output** with smooth coherent results.
However, it is sensitive to noise (CIH limitation) and it is not robust with large displacement (regularization term).
This is ususallyy used for object tracking.

The result is obtained via Lagrange multiplier with minimization on u and v;

$$J = \int\int_{\mathscr{R}} (uf_x + vf_y + f_t)^2 + \lambda(\|\nabla u\|^2 + \|\nabla v\|^2) dxdy$$

this is done by recalling that

$$\int\int_{\mathcal{R}} \Theta(x,y,w,w_x,w_y) dxdy$$

is optimized by imposing

$$\frac{\partial\Theta}{\partial w} - \frac{\partial^2\Theta}{\partial x \partial w_x} - \frac{\partial^2\Theta}{\partial y \partial w_y} = 0$$

with $w = u$ and $w = y$ respectively we get

$$\begin{matrix} \lambda\nabla_2 u = (uf_x + uf_y + f_t) f_x \\ \lambda\nabla_2 v = (uf_x + uf_y + f_t) f_v \end{matrix}$$

$\nabla_2 z$ can be approximated as $\overline{z} - z$ where $\overline{z}$ is the average of local values

$$\begin{cases}\lambda(\bar{u}-u) &= (uf_x+vf_y+f_t)f_x \\ \lambda(\bar{v}-v) &= (uf_x+vf_y+f_t)f_y\end{cases} \quad \begin{cases}(f_x^2+\lambda)u+f_yf_tv &= \lambda\bar{u}-f_tf_x \\ f_xf_tu+(f_y^2+\lambda)v &= \lambda\bar{v}-f_tf_y\end{cases} \quad \begin{cases}u &= \bar{u}-f_x\frac{\bar{u}f_x+\bar{v}f_y+f_t}{\lambda+\|\nabla f\|^2} \\ v &= \bar{v}-f_y\frac{\bar{u}f_x+\bar{v}f_y+f_t}{\lambda+\|\nabla f\|^2}\end{cases}$$

Final results

Horn shunk proposed the iterative algorithm.

□

7.2) Block Matching Method

This method allows to use discrete signals.It uses a support of a rectangular block of pixels. One motion vector is generated per block. The output is a displacement field, **motion vector field**

- Affine methods: 6 parameters per block, also represents zoom and not only translation

This technique is very popular as it gives good results at a low computational cost.

Consider a $P \times Q$ block of pixels inside a $N \times M$ image. The block starts at indexes p, q:

$$B_{p,q} = \{p, p+1, \dots, p+P-1\} \times \{q, q+1, \dots, q+Q-1\}$$

And the luminance vector at time k:

$$f_k(B_{p,q}) = [f(p,q,k), \dots, f(p+P-1, q+Q-1)]^T$$

The block matching method consists in computing the **dissimilarity between blocks and selecting those with minimum dissimilarity**. That is

$$(\hat{i}, \hat{j}) = \arg\min_{i,j} d[f_k(B_{p,q}), f_h(B_{p-i,j-q})]$$

In general we have a **forward motion**, that is $h = k-1$ with h the current frame and k the reference frame. Therefore the OF field at frame h will show the direction in which the blocks will be at frame k.

$$\forall (n,m) \in B_{p,q}, (u_{h\to k}, v_{h\to k}) = \arg\min_{(i,j)\in\mathcal{W}} d[f_k(B_{p,q}), f_h(B_{p-i,q-j})] = \arg\min_{(i,j)\in\mathcal{W}} J(i,j)$$

where d or J is the minimization criterion and $\mathcal{W}$ is the set of candidate pixels.

Measures

One valid **quality** measure is the **energy of the prediction error** (to PSNR), that is the MSE of the predicted block and the real block. That is:

$$e(n,m) = f_k(n,m) - \tilde{f}_k(n,m) \to \mathscr{E} = \frac{1}{NM}\sum_{n,m} e^2(n,m) \to PSNR = 10\log_{10}\frac{255^2}{\mathscr{E}}$$

where $\tilde{f}_k(n,m) = f_h(n+u_{h\to k}, m+v_{h\to k})$ is the predicted image from the motion vector field

A **cost** measure is the **coding cost**, that is, the number of bits to losslessly encode the motion vector field, since it depends on the implementation, the general empirical entropy is used.

Finally the **performance** measure is the **computational complexity**. The following choices impact the performance:

- Block size: determines the number of blocks:
- Number of candidate vectors ($i, j \in \mathcal{W}$)
- Cost function d

Cost Function

The **cost function** of $d()$ is based on $\mathcal{L}_1$ norm (SAD) or $\mathcal{L}_2$ norm (SSD). With SSD the one with smallest MSE but more computing cost

But a regularization term is also added, so the minimization is on J:

$$J(\mathrm{v}) = d(\mathrm{v}) + \lambda_{ME} r(\mathrm{v})$$

where λ_{ME} affects the importance of the regularization term.
Possible regularization terms are:

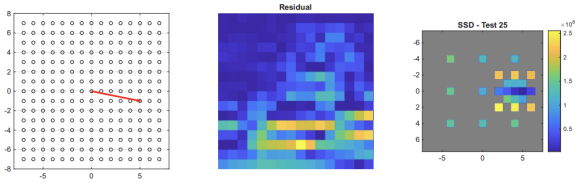
- Cost function: choose not best MSE, but best coding option.
- Distance: prioritize vectors with length same as mean of adjacent blocks

Cost Function	Pro	Con
SSD	Optimizes PSNR by minimizing energy in error	Larger rate (outliers), harder to compute
SAD	Better rate (implicit regularization), stronger to outliers	Worse PSNR
SAD+regularization	Best option $J_{reg} = \mathcal{L}_p + \lambda R$	

Candidates

The **number of candidates** can also be reduced by using some research strategies:

- Naive: test every vector i, j
- Less naive: test every vector in a $2A + 1 \times 2B + 1$ window center in i, j
- Three Steps Search (3SS): Assumption that error function is unimodal (single global minimum and no local minimum) Test 4-8 points and choose minimum error, now divide window and search 4-8 with window centered in previous minimum repeat

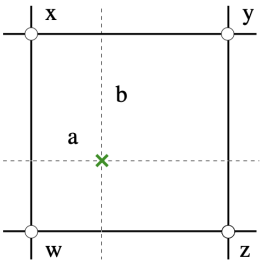


Example

- Diamond Search: search in 9 point diamond pattern, then extend pattern in direction of minima
- Hexagon Search: same as diamond but with hexagon (more modern)
- TZSearch: new technique that adaptively changes. Start with big block, if error is too big split it, repeat.

It is also possible to test sub-pixel positions by interpolation:

$$f(n + a, m + b) = (1 - a)(1 - b)x + a(1 - b)y + (1 - a)bz + abw$$



Example

Fixed Search	Unbound Search
Fixed number of steps	Iterative, faster
Guarantees global minimum	Can get stuck in local minima

Block Size

A **large block size** reduces complexity (less blocks), less coding cost, increased MSE. The ideal block size is 16×16.

Small Blocks	Big Blocks	Variable Size
Better Precision	Worse Precision	Better precision
Worse complexity	Better complexity	More complex (if required)
More coding cost	Less coding cost	More costly (if required)

7.3) Parametric Methods

A parametric model shows the motion field as a closed form function of the pixel position. The dof are the parameters of the function. It can be global or region based (local).
Block Matching is a special case of parametric methods with 2 parameters per bock (u,v translations), that is the **translational**:

$$v(p) = \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

The affine model is

$$v(p) = b + Bp = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} + \begin{bmatrix} b_3 & b_4 \\ b_5 & b_6 \end{bmatrix} p$$

This has only 6 dof but can represent many complex fields: rotation, zoom, translation

How are the parameters of the model estimated?

Indirect Estimation
First dense field, then find global motion by least squares.

$$\pi^* = \arg\min_{\pi} \sum_{n,m \in \mathcal{R}} [u(n,m) - u_\pi(n,m)]^2 + [v(n,m) - v_\pi(n,m)]^2$$

where $\pi = [b_1 \dots b_6]$ is the parameter set and $\cdot_\pi$ the dense field and $\cdot$ the field

Direct Estimation
Use the parameters directly in the estimation:

$$\pi^* = \arg\min_{\pi} \sum [u_\pi f_x + v_\pi f_u + f_t]^2$$

Or minimize the SAD or SSD

$$\pi^* = \arg\min_{\pi} \sum [e]^2 \qquad e(n,m) = f(n - u_\pi(n,m), m - v_\pi(n,m), t-1) - f(n,m,t)$$

SAD is a specific parametric affine estimation with $B = 0$

7.4) Deep Learning Methods (TODO)

Paradigm shift: classical (variational / block-based) use hand-crafted models; DL treats ME as a **supervised or unsupervised** learning problem, usually via **CNNs**. Needs specialized architectures for pixel-level correspondence, large datasets, and GPUs.

Ground-truth challenge: dense OF labels are hard to get for real video ⇒

- **synthetic datasets** (FlyingChairs, Sintel) for perfect labels,
- **data augmentation** (geometric + photometric),
- **unsupervised** training: minimize the **photometric error** between current frame and warped reference.

Architecture	Year	Key idea	Note
FlowNet	2015	first end-to-end CNN for OF (FlowNetS simple, FlowNetC correlation layer)	weak on small displacements / repetitive texture; ~10–100 FPS
FlowNet 2.0	—	stacked FlowNets + warping	big accuracy gain, higher complexity
PWC-Net	2018	**P**yramid + **W**arping + **C**ost-volume	efficient, hardware-friendly
RAFT	2020	**recurrent GRU** updates on a high-res flow field; all-pairs **correlation pyramid**	SOTA; strong zero-shot generalization; iterations tunable (speed vs. quality)

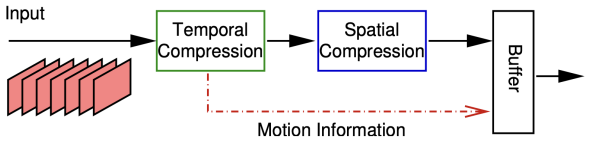
DL **dominates analysis** (optical flow); integration into **real-time video coding** still trades accuracy against practical constraints. Strengths: robustness to occlusions and large displacements, high precision. Open issue: **generalization** drops on data far from the training distribution.

7.5) Wrap-up

- ME extracts motion from video; it targets **optical flow**, not physical motion (other sensors handle the latter).
- **Variational / H&S:** dense, smooth, physical; noise-sensitive, weak on large displacements.
- **Block matching:** conceptually simple, the workhorse of **video compression**; criterion (SSD/SAD/reg.) × search strategy (FS/3SS/diamond/hexagon/TZ) × block size.
- **Parametric:** compact (translational 2 / affine 6 params); BM = translational special case ($B = 0$).
- **Deep learning:** emerging, very effective for analysis tasks (FlowNet → RAFT).

8) Video Coding Principles

Video compression uses the spatial redundancy (seen in jpeg) but also time redundancy via motion fields



General Scheme

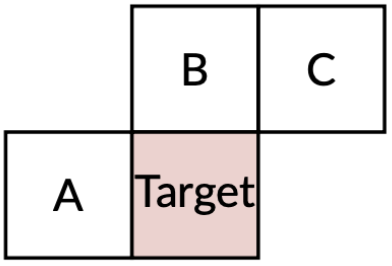
The temporal compression works by dividing the image in blocks $B_k^{(p)}$ and finding the most similar block $B_h^{(p+v)}$ in the ref image. The resulting displacements form the motion field.

The resulting vector fields are similar to the geometry of the scene, so the signal is NOT sparse. Therefore Exp-Golomb should **not** be used (not centered in 0).

We can still use predictors:
To reduce the bitrate we exploit the correlation between adjacent vectors. We define a Motion Vector Predictor (MVP) and encode only the Difference (MVD)

$$MVD = MV - MVP$$

One useful predictor is the median amongst 3 adjacent blocks:



Median Blocks

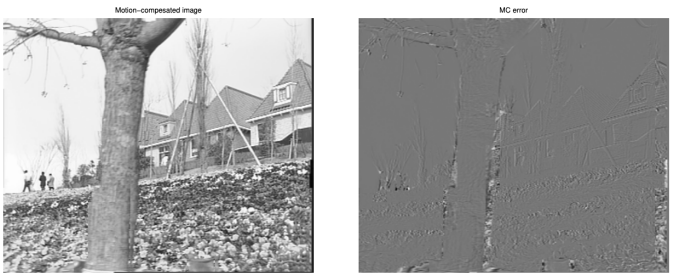
This reduces substantially the number of bits needed.

By also doing prediction (median) before entropy encoding we can further increase the compression.

Motion Compensation

We can predict the entire new image by copying the blocks in the new position:

$$\hat{I}_k(p) = I_h(p + v^*(p))$$



Example

But not all blocks should be replaced so they are signaled as intra (use og block) or inter frame blocks (use predictor).

For each $B_k^{(p)}$ do:

- Perform Motion Estimation (ME) with ref image h:

$$J(v) = d(B_k^{(p)}, B_h^{(p+v)}) + \lambda_{ME} r(v) \rightarrow v^* = \arg\min_v J(v)$$

- **Mode select**: decide if inter or intra (see 7.3) Mode Selection)
  - If intra use jpeg
  - If inter encode v^* and $E(p) = B_k^{(p)} - B_h^{(p+v)}$

A variable block size is more effective although it is more complex to implement.

Recap of the design parameters:

- **Block Size/Shape:** Small block: high PSNR, higher rate and time. Variable most effective
- **Cost function d:** SSD optimizes PSNR but higher rate (outliers), SAD better rate (implicit regularizaton)., SAD+regularization best option
- **Search:** full best vector but very high complexity, fast close to optimal but much less complex
- **Motion:** translational 2 parameters, affine 6 parametes, more complex some times better RD

Advanced motion compensation:
A fram can be predicted from many different frames together

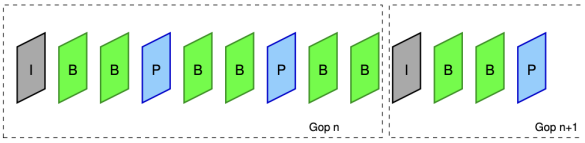
- Multiple Reference Frames: Slices can maintain one (P) or two (B) lists of reference images kept in the Decoded Picture Buffer (DPB)
- Generalized B slices: The encoder can flexibly select predictors from these lists and combine them using weighted averages

8.2) Group of Pictures (GOP)

Each frame can be one of the following three:

- I Frame: Intra coded, no prediction
- P Frame: Predictive coded, Inter frame
- B Frame: Bi directional prediction

The frames are organized into a periodical structure called Group of Pictures (GOP). This structure shows **how frames can be predicted from other frames**. In older standards I,P frames are called Anchor Frames (AF).



Example

MPEG-1, MPEG-2

- First frame of GOP is **always** I frame
- Between two AF there are some B frames (possibly also zero)
  GOP structure defined by I frames (GOP period) and number of B frames

I Frames are JPEG like coded:

- Low complexity and rate
- Can be decoded independently from other frames
- I frames are used for fast forward
- Stop error propagation
- Must have high quality as GOP depends on this frame

P Frames are predicted from previous AF. They have therefore a higher complexity but a higher compression rate per same block

B Frames have very high complexity (double ME) but also very high compression ratio.

8.3) Mode Selection

The mode selection problem is to decide how to encode each block or frame. Until H.264 the blocks were called macroblocks (MB), now they are coding units (CU).

4 Types of coding modes for each CU (mode selection problem)

- **Intra:** No temporal prediction, available for all frames
- **Inter:** ME/MC prediction. Not on I frames
- **Direct:** Motion vector from last frame, no additional coding. Not on I frames
- **Lossless:** available for all frames

Moreover since CUs can be of varying sizes we also have a **block partition problem.**

The solution is very much a brute force one:
Suppose fixed block size. The aim is to find the coding mode i_k that minimizes D while having rate R. Quantization step is given

$$D = \sum_{k=1}^{K} D_k(i_k, Q) \qquad R = \sum_{k=1}^{K} R_k(i_k, Q)$$

Therefore we must minimize

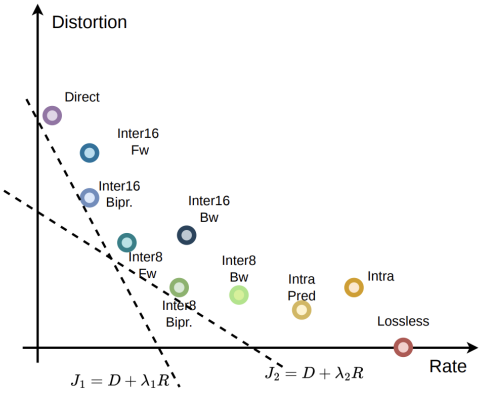
$$J(i, Q, \lambda) = \sum_{k=1}^{K} D_k(i_k, Q) + \lambda \sum_{k=1}^{K} R_k(i_k, Q) = D + \lambda R$$

Too complex, therefore we just minimize each block independently

$$J_k(i_k, Q, \lambda) = D_k(i_k, Q) + \lambda R_k(i_k, Q)$$

The parameter λ is determined empirically for each coded and also $\lambda_{ME} = \sqrt{\lambda}$
The J_k equation can be seen as a line in the R-D plane and the solution is the first point that is touched by the line.

- High λ: Lower bit budget, will choose points with less rate and more distortion (direct)
- Low λ: Higher bit budget, will choose points with less distortion and more rate (lossless)



Example

For a quantization step Q an optimal λ exists, it is measured empirircally:

- MPEG-2: $\lambda = aQ^2 + b$
- H.264: $\lambda = c2^{dQ+e}$

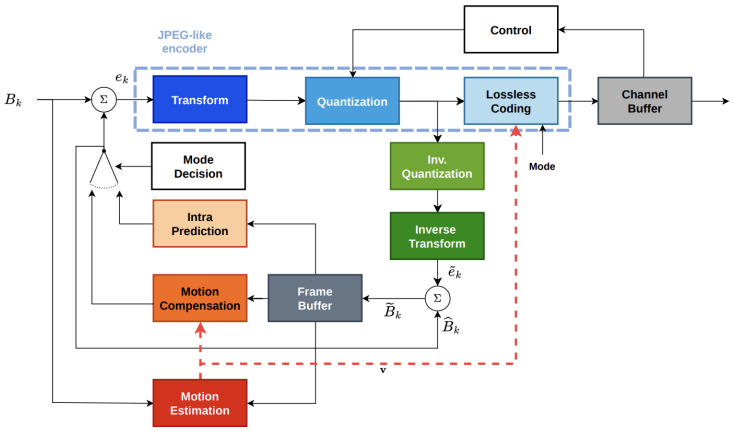
Also the **lock partition problem**, that is selecting the best variable block size.
We start with largest block, we divide into sub blocks if $J_{split} = \sum J_{subblock,i} < J_B$. When we achieve a better performance we block the search so to reduce complexity (suboptimal but reasonable results)

8.4) Hybrid Video Codec

The video coder implements mode selection only in encoder:

- **Null predictor:** compress entire frame (DCT+Q)
- **Temporal Prediction:** via MC/ME
- **Space or Intra:** predict entirely from encoded blocks in the same frame

This is the full video encoder:



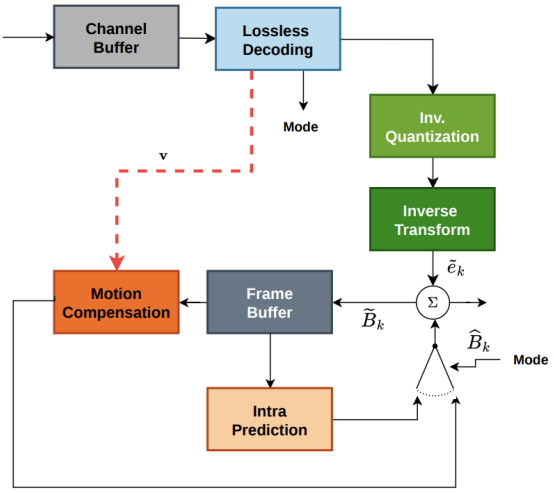
Video Encoder

The block B_k is encoded as follows:

- 🔵 The residual error $e_k = B_k - \hat{B}_k$ is found and encoded in a jpeg like format. Also the mode is encoded.
- 💚 Local decoder: find $\tilde{B}_k = \tilde{e}_k + \hat{B}_k$. This is how the block will be decoded and it is saved in frame buffer (if inter).
- 🧡 Predictors: ME/MC predictors (based on B_k+buffer) or Intra predictors. This predicts
- ⚪ Control: Final generated bits are saved in channel buffer. If the rate is too high, the control block expands the quantization step. The buffer is written at speed R_C and read at speed R_T.

- If $R_C > R_T$: the buffer grows, if it surpasses a threshold the controller increases the quantization step $\longrightarrow R_C \nearrow$
- If $R_C < R_T$: the buffer empties, if it decreases below a threshold the controller reduces the quantization step $\longrightarrow R_C \searrow$

The decoder is a subset of the encoder!



Decoder Is Subset After Lossless Decoding

The decoder is a simplified version without ME, mode decision, partition, control.
Each block is decoded based on the mode, however the modes are not standardized, just their syntax. The encoder could take a suboptimal approach, the decoder doesn't care.

9) Modern Video Compression Standards

The decoder is the only part defined in the standard:
Standards exclusively define the bitstream syntax and the decoding process. Encoder architecture and optimization strategies remain open problems for industrial competition.

The improvements come from how many ways the encoder can encode the images. This is mainly related to how the image can be partitioned in blocks.

Feature	H.264/AVC	H.265/HEVC	H.266/VVC	AV1
Base unit	Macroblock 16×16	CTU up to 64×64	CTU up to 128×128	Superblock up to 128×128
Partitioning	VBS: 16×8, 8×16, 8×8, 4×4 sub-MB	Quad-tree only (CU → PU + TU, independent trees)	QT + MTT: binary (1:1) and ternary (1:2:1) splits; CU/PU/TU unified	10-way split incl. 4:1 and 1:4 rectangular; recursive
Intra prediction	16×16: 4 modes; 4×4: 9 modes (8 dir + DC)	35 modes (33 dir + DC + Planar)	65 dir + wide-angle for non-square blocks	Similar to VVC
Intra mode signaling	—	MPM list: 3 candidates	MPM list: 6 candidates	—
Inter MV precision	Integer pel (+ H.264 uses $1/2$ pel)	$1/4$ pel, 6-tap interpolation filter	$1/4$ pel (same as HEVC)	$1/8$ pel, selectable filters (sharp/smooth)
MV prediction	Skip/Direct: median of spatial neighbors; 0 bits if correct	Merge mode: 5 spatial + 1 temporal candidates	Merge mode (same as HEVC); affine (4- or 6-param)	Affine; compound (inter + intra in same block)
Motion model	Translational only	Translational only	Affine: rotate, zoom, shear	Affine; compound inter+intra
Reference frames	P-slices (1 list), B-slices (2 lists)	P-slices, B-slices; generalized B	Same + flexible DPB management	Multiple refs
Transform	DCT 4×4 (integer approx.)	DCT up to 32×32; DST for 4×4 intra	Multiple Transform Selection (MTS); non-square transforms	—
Quantization	QP: step doubles every $\Delta QP = 6$	Same	Same	Same
Entropy coding	CABAC (or CAVLC)	CABAC only	CABAC only	— (own arithmetic coder)
In-loop filters	Deblocking on 4×4 grid; adaptive strength	Deblocking on 8×8 grid + SAO (edge/band offset)	Deblocking + SAO + ALF + LMCS (HDR)	CDEF (directional) + Loop Restoration (Wiener) + Film Grain Synthesis
Bitstream format	Annex-B or Packetized NALUs; 1-byte header	Same; 2-byte header (Temporal ID, Layer ID)	Same as HEVC	OBUs (no start codes); length-prefixed only
Random access	IDR only	IDR + CRA (RASL) + BLA	Same	Keyframe-based
Parallelism	Slices	Tiles + WPP	Tiles + WPP + subpictures	Tiles

---

H.264/AVC has 16×16 MB or 8×8 and 4×4 MB which are inefficient for 1080p or 4K videos

Motion Vectors are predicted using spatial neighbor. If median is correct, 0 bits are used

uses DCT for 4×4 blocks

Lossless coding is done via Context Adaptive Binary Arithmetic Coding (CABAC)

Deblocking is used: Analyzes edges between 4×4 blocks. Filtering strength adapts dynamically to the coding mode, motion vectors, and quantization step of neighboring blocks

H.265/HEVC has a Coding Tree Unit (CTU) where a MB of 64×64 can be divided into other squares until 4×4

HEVC uses Coding Tree Unit (CTU) with quad tree split. Each CTU gets recursively split into CUs (divide by 4) and each CU can be divided into Predictions Units (PU) or Transform Units (TU). These (PU,TU) are independently created from eachother since a block that works great fro PU might not work for TU

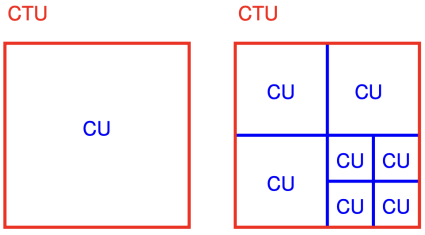
HEVC also introduces quarter pixel fractional precision. Uses 6-tap filter to interpolate sub pixels

MV is predicted via **Merge Modes** use dynamic candidate list: aggregates 5 spatial (adjacent blocks) and 1 temporal candidates, shares MV of best candidate and ref frame index. Also in VVC

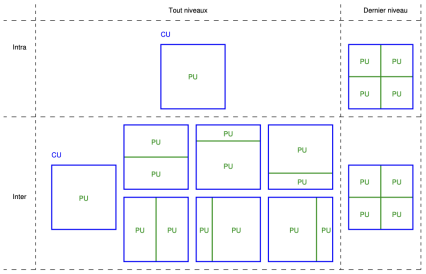
uses DST for 4×4 blocks

the quantization step size doubles for every increment of 6 in the Quantization Parameter (QP)

Uses deblocking + Sample Adaptive Offset (SAO): categorizes pixels (edges, bands) and adds offsets to correct ringing artifacts



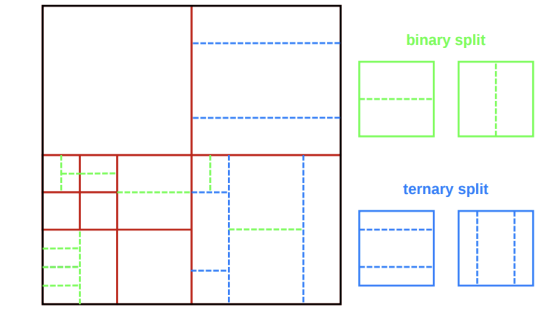
Example



Example

H.266/VVC has blocks of up to 128×128. New splits are defined (binary 1:1 or ternary 1:2:1 splits). The sub pixel prediction is increased to $1/8$. From here also the blocks are not rigid, they can rotate, zoom, shear. uses 4 or 6 parameters. Uses **Multiple Transform Selection (MTS)** for rectangular blocks

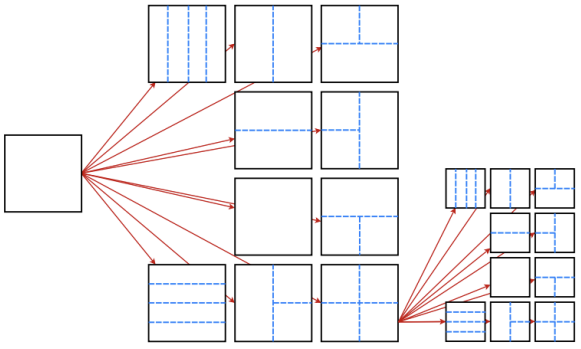
Adds the Adaptive Loop Filter (ALF) and Luma Mapping with Chroma Scaling (LMCS) specifically designed to preserve details in HDR (High Dynamic Range) video



Example

AV1 can create up to 10 different splits. Allows to combine inter with intra predictions in same block

Many post processing effects: Directional filters for smooth edges, Loop restoration filters for large windows, Film grain synthesis



Example

Trying every combination is computationally unfeasable:
Depth First Search (DFS) is used to explore efficiently block geometry:

- Top Down Exploration: try as a whole, then split
- Causal Dependency: Z scan order for subblocks, blocks cannot be predicted until top and left are reconstructed
- Inner Loop (prediction): for each block test inter/intra modes to find local minimum
- Bottom up: compare costs $\sum J_{children} < J_{parent}$

fats algorithms and ML based ones can prune the tree faster

- Texture analysys: if a texture has low variance (smooth surface) then small blocks are bypassed
- Temporal correlation: search space by exploiting the partition depth of the co-located CU in the reference frame.
- ML Classifiers: lightweight predict splits from pixel features

Cost Of Freedom

Recall this

H264	H265	H266
4 predictors for 16 $\times$ 16 blocks. 9 types (8 directions+DC) for 4 $\times$ 4	33 modes (33 directions +DC+planar)	65 directional modes + wide angle modes for non square blocks

Notice that the bits needed to encode a block depends on the generators P:

$$P \propto 2^B$$

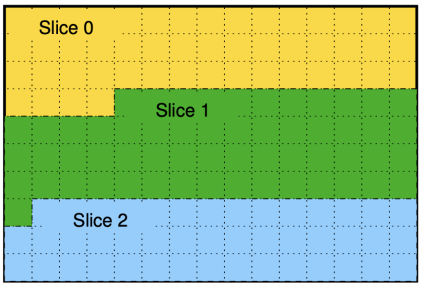
VVC needs 7 bits for 67 modes. Sending 7 bits per block is inefficient since the rate explodes when there are small blocks!

Use **Most probable Mode (MPM)**, the encoder predicts mode using spatial context. List of MPMs s built (HEVC has 3, VVC has 6). So these are encoded with small codewords

9.2) Network Abstraction and Parallelism

In network transmission, the data is subject to possible bit errors and packet losses. One bit error can propagate for various frames and packet losses can block the video stream.

Picture is divided into slices so that CABAC is reset for each slice



Example

From h.264 the video standard for compression and transport is created:

- **Video Coding Layer (VCL):** video codec+slices generation
- **Network Abstraction Layer (NAL):** Encapsulates VCL, adds metadata and transports

Works both for RTP/IP communications and also ISO or diffusion

The slices are encapsulated into **VCL NAL Units (NALU)** or Non-VCL NALU

- **VCL NALU:** Modes, MV, quant coeff
- **Sequence Parameter Set (SPS):** important encoding params: size, color, etc
- **Picture Parameter Set (PPS):** per picture coding optons
- **Supplemental Enhanced Info (SEI):** subtitles, HDR, user data
- **Access Unit Delimiter (AUD):** optional NALU signaling

The (logical) group of all NALUs to decode a frame is called **Access unit (AU)**. Delivered either via bitstream or packets

- **Byte Stream h.264:** Inject start code before NALU, use emulation prevention to avoid bitstream to have start code. Good for continuous streaming, adds cpu overhead
- **Packets h.265, h.266:** A length field (+ temporal ID and Layer D) is written before every NALU, $O(1)$ memory jump but if there is a corruption fails

AV1 uses Open Bitstream Units (OBU)

Intermediate nodes, called Media Aware Network Elements (MANE), read the header (deep packet inspection) and can decide to drop B frames without touching important parameters if necessary. Moreover CRC is added to SPS to check for bit errors

If a NALU exceeds MTU, RTP layer slices the NALU into Fragmentation Units (FU) that will be reassembled

Resynchronization:
Decoding may restart at a Random Access Point (RAP) signaled by an Instantaneous Decoding Refresh (IDR), no pictures after IDR are referenced from pictures preceding it. This however reduces the compression efficiency

HEVC introduced Clean Random Access (CRA):

Tiles:
Tiling is used to create self contained slices with many CTUs. These allow for parallelization in encoding/decoding

Slices can also be divided into Wavefront Parallel Processing (WPP).

Type	What it is	What happens at decode time	When / why it's used
IDR	Intra frame that fully clears the DPB	All past references discarded; CABAC reset; any following frame can only reference frames after the IDR	Hard scene cuts; absolute stream start; guarantees clean restart but breaks temporal prediction across the boundary → compression loss
CRA	Intra frame that does *not* clear the DPB	If seeking: decoder starts here, discards RASL frames (their references are missing) and decodes the rest cleanly. If playing normally: RASL frames are decoded using pre-CRA references → no compression loss	Introduced in HEVC as a cheaper seek point than IDR; lets the encoder maintain long-range temporal references across the access point
BLA	A CRA whose leading RASL references have been destroyed by a network node	Decoder immediately drops all RASL frames without looking up references; rest decoded cleanly	Generated by transcoders/packagers/ad-splicers when they cut and stitch streams at a CRA; they flip the NALU header CRA → BLA to signal "leading frames are broken"
WPP	Encoding/decoding CTU rows in parallel with a fixed 2-CTU stagger between adjacent rows	Row $k+1$ can start as soon as 2 CTUs of row k are done; the top-left CABAC context needed for spatial prediction is always available by then	Finer-grained parallelism than tiles (no hard independence boundary, so no MV restriction); slight compression penalty vs. fully sequential encoding

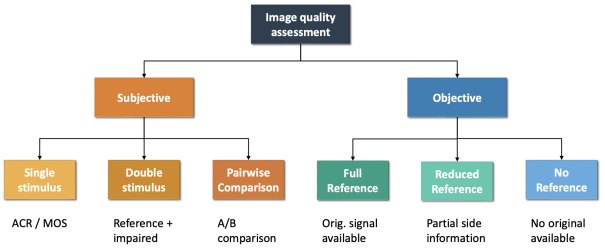
10) Audio and Speech Compression (TODO)

11) Quality Assessment and Quality of Experience for Multimedia Services

Quality assessment tries to give quantitative feedback by a signal perceived by a human.

- **Qualitative:** subjective quality assessment
- **Quantitative:** give a numeric value

Good for cost-benefit analysis: rate is objective but does not necessarily reflect quality of experience



taxonomy

11.1) Subjective Quality Assessment

Most reliable way to measure quality of experience, however it is costly and time consuming and requires many users to provide reliable results

- **Laboratory equipment,** including screen and viewing distance, illumination and reverberation characteristics of the room.
- **Data set,** including the original and processed contents and their distribution
- **Test methodology,** including the rating target (quality, comparison, or impairment), the scale (categorical or continuous) and the type of stimuli (single or double).
- **Score processing,** including score normalization, outlier detection, mean score and confidence intervals computation, and significance tests.

Fixed viewing angles and monitors and resolutions are selected.

Prior to a test a screening for visual acuity or normal color vision is performed. They should also be informed of the methodology. A session should not last more than 30 min as they might chaneg sensitivity or opinions too much in that time frame: run train with all possible types of stimuli to inform participants, keep session short, pay them and randomize stimuli

The type of stimuli depends on the goal, video stimuli are used to compare spatial (Spatial Information SI, textures) and temporal predictions (Temporal Information TI, action)

- **Spatial Information (*SI*):** A measure that indicates the amount of spatial detail in a picture. It is usually higher for more spatially complex scenes. Each image is fintered with **sobel filter** that highlights contours. The STD is computed and max value is chosen
- **Temporal information (*TI*):** a measure that indicates the amount of temporal changes of a video sequence. It is usually large for high-motion sequences. Measured as difference of pixel values at same position between two frames. The STD is computed and the max is selected

Single Stimulus (SS) or Absolute Category Rating (ACR): single image or video shown (also possible with reference). During the data analysis, a differential mean opinion score (DMOS) of quality will be computed between each test sequence and its corresponding (hidden) reference.

In **Double Stimulus Impairment Scale (DSIS)** users rate the degradation. See og, then watch compressed and grade

in **Pair wise Comparison (PC)** the user must choose between og or compressed without nowing which is which. Results stored in comparison matrix where $c_ij = n$ is the amount of times that c_j was preferred over c_i

$$C = \begin{bmatrix} c_{00} & c_{01} & c_{02} & \dots \\ c_{10} & c_{11} & c_{12} & \\ c_{20} & c_{21} & c_{22} & \\ \vdots & & & \ddots \end{bmatrix}$$

Finally it results that PSNR is not sufficient. Also vieweing condition and smoothness impact performance

Reliable Score:
Define votes as random process: estimate mean ($m = \frac{1}{N}\sum x_i$) and variance ($\sigma = \sqrt{\frac{1}{N}\sum(x_i - m)^2}$) (small= mostly agree on mean value). Then a confidence interval around the mean is computed so to have 95% confidence to contain the true mean. This is

$$ci = [-1.96 \cdot SE, 1.96 \cdot SE] \text{ with } SE = \sigma^2/\sqrt{N}$$

Method	Reference Visible?	Speed	Accuracy	Typical Use Cases
ACR (Absolute Category Rating)	No	High	Medium	Large-scale streaming evaluation
ACR-HR	Hidden reference	High	High	Compression quality studies
DSIS (Double Stimulus Impairment Scale)	Yes	Medium	Very High	Codec validation
PC (Pairwise Comparison)	Relative comparison	Low	Excellent	Fine perceptual ranking

---

It is possible to crowdsource this, but with no controlled viewing conditions

11.2) Objective Quality Assessment

Quality assessment algorithms usually predict visual quality by comparing a distorted signal against a reference, typically by modeling the human visual system. It is based on mathematical models that try to replicate human perception

- Full reference metrics: PSNR, SSIM, VMAF
- No-reference metrics: NIQE, BRISQUE, CLIP-IQA
- Reduced reference metrics: RRED

Signal is $I(n, m, c, t)$ and processed signal si $\hat{I}(n, m, c, t)$. That is component c at position n, m at time t.

$$MSE(t) = \frac{1}{NM}\sum_{n,m}\Big(I(n,m,1,t) - \hat{I}(n,m,1,t)\Big)^2$$

$$PSNR(t) = 10\log_{10}\frac{255^2}{MSE(t)} \approx 48dB - 10\log_{10}MSE(t)$$

PSNR only measures pixel difference, not visual quality
Structural Similarity Index (SSIM) is closer to human quality evaluation (more sensitive to structures, edges, contrast)

$$SSIM(x,y) = l(x,y)^\alpha c(x,y)^\beta s(x,y)^\gamma$$

where

- l is luminance change

- c is contrast change
- s is structure comparison

Video distortion produces RD curves: **Bjontegaard metrics** can be used to compare these curves

- Delta Rate: it is the average rate difference at the same quality, expressed as %
- Delta PSNR: it is the average PSNR difference at the same rate

12) Adaptive Streaming